

Manual de negocio

DataMart 360

Sistema de Gestión y Análisis de Datos para una Cadena de Supermercados



Autora:

Andraida De la cruz

Tecnóloga Superior en Desarrollo de Software

Año: 2026

Índice

Introducción.....	6
Objetivos	7
Objetivo general.....	7
Objetivos específicos	7
Alcance.....	8
Información General de la empresa.....	9
Misión:	10
Visión:.....	10
Valores	10
☐ Eslogan:	10
☐ Ciudades:	10
☐ Sucursales:.....	10
Metodología del proyecto.....	11
Arquitectura de la solución.....	13
MÓDULO 1	14
Modelo de negocio.....	14
Estructura organizacional	17
Procesos del negocio	19
Reglas del negocio.....	24
Glosario del negocio.....	28
Indicadores Clave de Desempeño (KPIs).....	31
Diccionario del Negocio.....	34
Catálogo Maestro	35
Matriz de Requerimientos Funcionales y No Funcionales	37
Casos de uso del negocio.....	39
MÓDULO 2	40
Diseño de Base de Datos.....	40
Lista oficial de entidades.....	41

Catálogo de Restricciones lógicas (Constraints).....	44
MODELO CONCEPTUAL.....	45
MODELO LÓGICO	46
MODELO RELACIONAL	47
MÓDULO 3	48
Implementación PostgreSQL	48
Script SQL completo.....	48
MÓDULO 4	59
Población y Gestión Inicial de Datos	59
MÓDULO 5	64
SQL Nivel básico	64
Nivel Intermedio	71
Nivel Avanzado.....	80
MÓDULO 6	89
Administración y Optimización de PostgreSQL	89
MÓDULO 7	97
Excel para Análisis de datos	97
Introducción	97
7.2 Limpieza y preparación de datos.....	98
7.3 Power Query	98
7.4 Funciones de búsqueda y referencia	99
7.5 Funciones de análisis	100
7.6 Análisis mediante tablas dinámicas	100
7.7 Visualización y exploración de datos.....	101
7.8 Formato condicional	101
7.9 Validación de datos.....	102
7.10 Dashboard ejecutivo en Excel.....	102
Dashboard final	104
Conclusiones del módulo	104

MÓDULO 8	105
POWER BI.....	105
Modelo de datos en Power BI	108
Validación del modelo	109
Medidas DAX principales.....	109
Página 1 — Resumen Ejecutivo.....	113
Página 2 — Análisis de Ventas	115
Página 3 — Clientes y fidelización	117
Página 4 — Productos	118
Página 5 — Inventarios y Stock.....	119
Página 6 — Compras y Proveedores	120
Página 7 — Operaciones y Sucursales	122
Botón de limpieza de filtros	123
Criterios de diseño	123
Validaciones realizadas.....	124
Conclusiones	125
Limitaciones	127
Fuentes de apoyo y herramientas utilizadas	129
Declaración sobre el uso de herramientas de inteligencia artificial	129

Introducción

En la actualidad, las organizaciones generan grandes cantidades de información a partir de sus operaciones diarias. En empresas del sector retail, datos relacionados con ventas, clientes, productos, inventarios, compras, proveedores y sucursales pueden utilizarse para conocer el comportamiento del negocio y facilitar la toma de decisiones.

Sin embargo, disponer de datos no es suficiente. Es necesario organizarlos, relacionarlos y transformarlos en información que pueda ser consultada y analizada de manera eficiente. En este contexto surge DataMart 360, un proyecto desarrollado para representar una solución de gestión y análisis de datos aplicada a una cadena ficticia de supermercados.

El proyecto comprende diferentes etapas, comenzando con el análisis de las necesidades del negocio y el diseño de la estructura de información. Posteriormente, se desarrolló una base de datos relacional en PostgreSQL, se generaron e incorporaron datos simulados y se realizaron consultas SQL para explorar y validar la información.

Como parte del proceso de análisis, los datos fueron transformados y preparados mediante Power Query y posteriormente utilizados en Excel y Power BI. En Excel se desarrollaron modelos, tablas dinámicas e indicadores, mientras que Power BI se utilizó para construir un dashboard interactivo compuesto por diferentes páginas de análisis y medidas DAX.

De esta manera, DataMart 360 integra conocimientos de bases de datos, SQL, análisis de datos, Excel y Business Intelligence, permitiendo recorrer un flujo completo desde el almacenamiento y procesamiento de los datos hasta su representación mediante indicadores y visualizaciones.

Objetivos

Objetivo general

Diseñar e implementar una solución integral de gestión y análisis de datos para una cadena ficticia de supermercados, utilizando una base de datos relacional en PostgreSQL y herramientas de análisis y Business Intelligence como Excel y Power BI, con el propósito de transformar datos operativos en información útil para el análisis del negocio.

Objetivos específicos

- Analizar los principales procesos y áreas de una cadena de supermercados que requieren almacenamiento y análisis de información.
- Diseñar un modelo entidad-relación y un modelo relacional que representen adecuadamente las entidades, atributos y relaciones del negocio.
- Implementar una base de datos relacional en PostgreSQL utilizando claves primarias, claves foráneas, restricciones y relaciones entre tablas.
- Generar e incorporar datos ficticios que permitan representar diferentes operaciones comerciales y administrativas de la empresa.
- Desarrollar consultas SQL para consultar, relacionar, validar y analizar la información almacenada.
- Implementar elementos de base de datos como vistas, funciones, procedimientos almacenados, triggers e índices.
- Utilizar Power Query para realizar procesos de transformación y preparación de los datos.
- Construir un modelo de datos en Excel utilizando Power Pivot, tablas dinámicas e indicadores.
- Desarrollar un modelo analítico en Power BI y crear medidas DAX para el cálculo de indicadores.
- Construir un dashboard interactivo que permita analizar ventas, clientes, productos, inventario, compras, proveedores y sucursales.
- Validar la consistencia de los datos y de los principales indicadores obtenidos durante las diferentes etapas del proyecto.
- Documentar el proceso de desarrollo, las herramientas utilizadas y los resultados obtenidos.

Alcance

El alcance de DataMart 360 comprende el diseño, implementación y análisis de una solución de datos orientada a representar los principales procesos de una cadena ficticia de supermercados.

El proyecto incluye el diseño de una base de datos relacional compuesta por 19 tablas, en las cuales se estructura información relacionada con clientes, productos, categorías, ventas, inventarios, compras, proveedores, sucursales, empleados, despachos, métodos de pago, promociones y fidelización.

Dentro del alcance se encuentra la implementación de la base de datos en PostgreSQL, incluyendo la creación de tablas, relaciones, restricciones, claves primarias y foráneas, así como diferentes elementos para el procesamiento y consulta de la información, entre ellos vistas, funciones, procedimientos almacenados, triggers e índices.

También se contempla la generación e incorporación de datos ficticios correspondientes al período 2024–2026, con el propósito de disponer de información suficiente para realizar pruebas y análisis.

En la etapa de análisis se incluye la utilización de Power Query, Power Pivot y Excel para la transformación, modelado y exploración de los datos mediante tablas dinámicas, indicadores y un dashboard.

Como producto final se desarrolla un dashboard en Power BI compuesto por siete páginas, orientadas al análisis de:

- Resumen ejecutivo
- Ventas
- Clientes y fidelización
- Productos y categorías
- Inventario y stock
- Compras y proveedores
- Operaciones y sucursales

El alcance también contempla la creación de medidas DAX, KPIs y visualizaciones interactivas para facilitar la interpretación de los resultados.

El proyecto está orientado principalmente a fines académicos, educativos y de demostración profesional, por lo que no contempla la implementación de la solución en una empresa real ni su conexión con sistemas empresariales productivos.

Información General de la empresa

Nombre: DataMart 360

Sistema de Gestión y Análisis de Datos para una Cadena de Supermercados

Logo:



Historia

DataMart 360 nació en el año 2007 en la ciudad de Quito, Ecuador, como resultado del sueño de Martín Herrera, un padre de familia con dos hijos que, tras quedarse desempleado, decidió invertir los pocos ahorros que tenía en un pequeño negocio de venta de productos de primera necesidad para su comunidad.

Desde sus inicios, Martín administró el negocio con disciplina, esfuerzo y una visión clara de crecimiento. Reinvertía gran parte de las ganancias en ampliar el inventario y mejorar el servicio, mientras mantenía un estilo de vida sencillo para fortalecer la estabilidad de la empresa.

Gracias a la confianza de sus clientes y al apoyo incondicional de su familia, el negocio logró un crecimiento sostenido. En 2009 se inauguró la segunda sucursal, marcando el inicio de una nueva etapa para la empresa.

A partir de 2010, DataMart 360 comenzó un proceso de expansión hacia diferentes ciudades del país, consolidándose como una cadena de supermercados enfocada en ofrecer productos de calidad, precios competitivos y una atención cercana a sus clientes.

Actualmente, la empresa cuenta con 12 sucursales y una bodega, distribuidas en cinco ciudades del Ecuador, manteniendo el mismo compromiso con el que inició: brindar a las familias una alternativa confiable para realizar sus compras diarias.

Misión:

Proporcionar productos de calidad a precios competitivos, garantizando una experiencia de compra cercana, confiable y eficiente que contribuya al bienestar de las familias y fortalezca la fidelización de nuestros clientes.

Visión:

Consolidarnos como la cadena de supermercados líder del país, reconocida por ofrecer productos de calidad, precios competitivos e innovación constante en el servicio al cliente.

Valores

Honestidad, responsabilidad, amabilidad, conciencia, respeto, Trabajo en equipo, Compromiso, Integridad, Orientación al cliente, Innovación y Transparencia.

- **Eslogan:** Calidad, ahorro y confianza para tu familia.
- **Ciudades:** Quito, guayaquil, cuenca, Riobamba, Latacunga.
- **Sucursales:** 13 incluida la bodega principal

Metodología del proyecto

El desarrollo de DataMart 360 se realizó mediante un flujo progresivo que permitió pasar desde el análisis del negocio hasta la construcción y validación de la solución de Business Intelligence.

1. Análisis del negocio

Identificación de las principales áreas y procesos que debía representar la solución:

- Ventas
- Clientes
- Productos
- Inventario
- Compras
- Proveedores
- Sucursales
- Operaciones

2. Diseño de la solución

Definición de la estructura general del sistema, las entidades necesarias y el flujo de trabajo de los datos.

3. Modelado de datos

Diseño del modelo entidad-relación y modelo relacional, estableciendo tablas, atributos, claves primarias, claves foráneas y relaciones.

4. Implementación en PostgreSQL

Creación de la base de datos y sus estructuras mediante PostgreSQL.

Se implementaron tablas, restricciones, relaciones, vistas, funciones, procedimientos, triggers e índices.

5. Inserción de datos

Generación e incorporación de datos ficticios para representar diferentes procesos comerciales y operativos de la empresa.

6. Consultas SQL

Desarrollo de consultas para validar, explorar y analizar la información almacenada en la base de datos.

7. Análisis con Excel

Transformación y preparación de los datos mediante Power Query, construcción del modelo de datos y desarrollo de análisis mediante tablas dinámicas, Power Pivot e indicadores.

8. Desarrollo en Power BI

Construcción del modelo analítico, creación de medidas DAX y desarrollo de las páginas del dashboard.

9. Validación

Comprobación de relaciones, cálculos, indicadores y resultados entre las diferentes etapas del proyecto.

10. Documentación

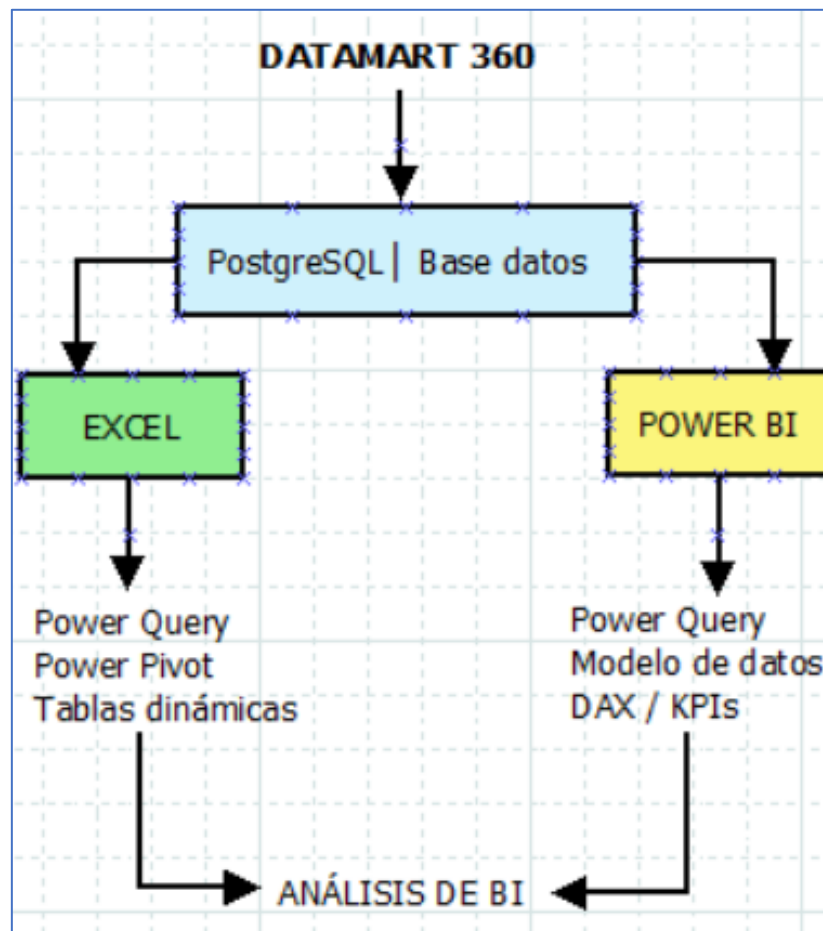
Registro del proceso de análisis, diseño, implementación, transformación, análisis y resultados obtenidos.

Arquitectura de la solución

Aquí recomiendo no utilizar el diagrama hecho con caracteres que tenemos actualmente en el README.

Para la documentación PDF sí debemos hacer una imagen real del diagrama.

La estructura conceptual que propones está bien, pero haría un pequeño ajuste:



MÓDULO 1

Modelo de negocio

1. Descripción del Modelo de Negocio

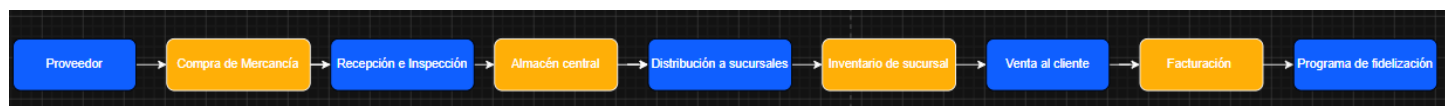
DataMart 360 es una cadena de supermercados orientada al comercio minorista (*retail*), dedicada a la comercialización de productos de consumo masivo para satisfacer las necesidades diarias de las familias ecuatorianas.

La empresa adquiere productos de proveedores nacionales e internacionales, los recibe en su centro de distribución principal, donde son inspeccionados, almacenados y posteriormente distribuidos a las diferentes sucursales. Cada sucursal administra su propio inventario y realiza las ventas directamente al consumidor final.

El modelo de negocio busca ofrecer productos de calidad a precios competitivos mediante una gestión eficiente del abastecimiento, el control del inventario y estrategias comerciales que promuevan la fidelización de los clientes.

2. Cadena de Valor

El funcionamiento operativo de DataMart 360 sigue el siguiente flujo:



Este proceso garantiza el abastecimiento continuo de las sucursales, el control del inventario y la correcta atención al cliente.

3. Componentes del Modelo de Negocio

El funcionamiento de la empresa se basa en los siguientes procesos estratégicos:

- Venta de productos de consumo masivo.
- Compra centralizada a proveedores autorizados.
- Recepción, inspección y almacenamiento de mercancía.
- Distribución interna desde el almacén central hacia las sucursales.
- Administración independiente del inventario de cada sucursal.
- Aplicación de promociones y descuentos comerciales.
- Programa de fidelización mediante acumulación de puntos.
- Facturación obligatoria de todas las ventas.

- Gestión y control del inventario para minimizar pérdidas.
- Comercialización mediante un alto volumen de ventas con márgenes competitivos.

4. Fuentes de Ingresos

Los ingresos de DataMart 360 provienen principalmente de la venta de productos pertenecientes a las siguientes categorías:

- Alimentos.
- Bebidas.
- Productos de limpieza.
- Productos de cuidado personal.
- Artículos para el hogar.

En el futuro, la empresa podrá incorporar nuevas categorías sin afectar su modelo operativo.

5. Público Objetivo

DataMart 360 está dirigido principalmente a:

- Familias.
- Personas de ingresos bajos y medios.
- Pequeños negocios que realizan compras minoristas.
- Clientes frecuentes afiliados al programa de fidelización.

6. Ventajas Competitivas

DataMart 360 busca diferenciarse mediante:

- Precios accesibles.
- Productos de calidad.
- Amplio surtido de productos de primera necesidad.
- Atención cercana y personalizada.
- Promociones permanentes.
- Programa de fidelización para clientes frecuentes.
- Gestión centralizada del abastecimiento que garantiza disponibilidad de productos en las sucursales.

7. Objetivos del Modelo de Negocio

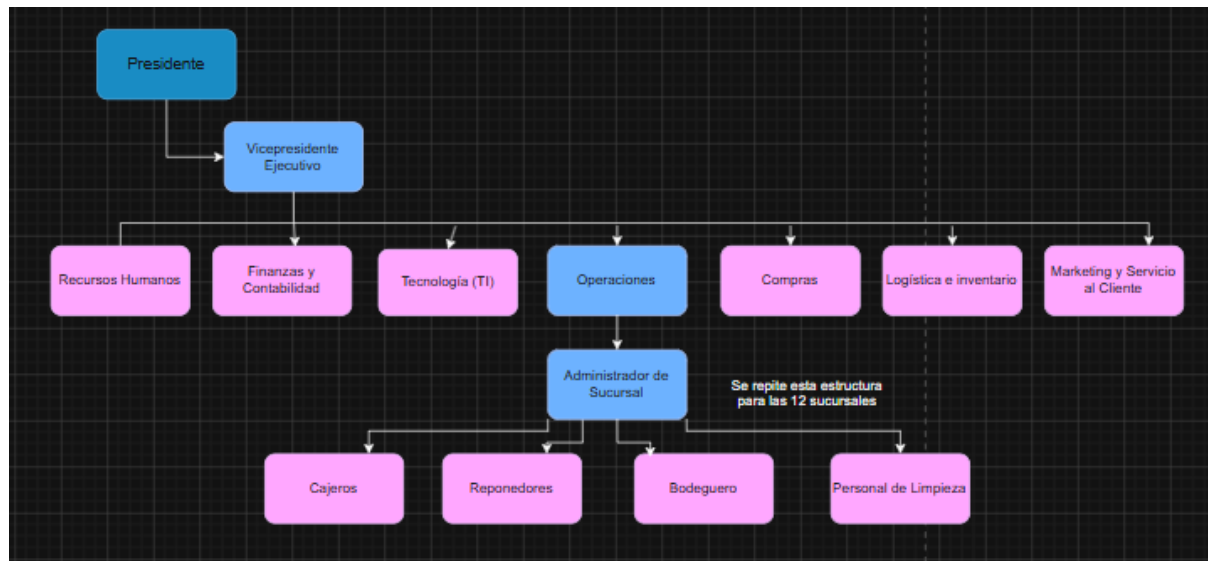
El modelo operativo de DataMart 360 tiene como finalidad:

- Maximizar la satisfacción del cliente.
- Garantizar la disponibilidad de productos.
- Optimizar la rotación del inventario.

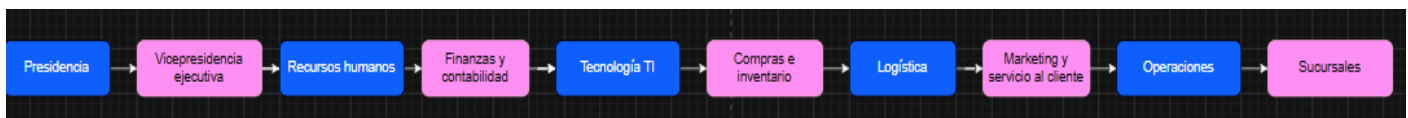
- Mantener una operación eficiente entre el almacén central y las sucursales.
- Incrementar la rentabilidad mediante una adecuada gestión comercial y operativa.
- Fortalecer la fidelización de los clientes.

Estructura organizacional

Organigrama organizacional:



Organigrama funcional:



Descripción de las áreas

✓ Presidencia

Define la estrategia general de la empresa, establece los objetivos corporativos y supervisa el desempeño global de DataMart 360.

✓ Vicepresidencia Ejecutiva

Coordina las diferentes áreas de la organización, asegurando el cumplimiento de los objetivos estratégicos y operativos.

✓ Recursos Humanos

Gestiona los procesos de reclutamiento, selección, contratación, capacitación, evaluación y administración del personal.

✓ Finanzas y Contabilidad

Administra los recursos financieros de la empresa, elabora los estados financieros, controla los presupuestos y supervisa los procesos contables.

✓ Tecnología de la Información (TI)

Administra la infraestructura tecnológica, los sistemas de información, la seguridad informática y brinda soporte técnico a todas las áreas de la organización.

✓ Compras

Negocia con proveedores, gestiona las órdenes de compra y garantiza el abastecimiento oportuno del almacén central.

✓ Logística

Coordina el transporte y distribución de mercancías desde el almacén central hacia las sucursales.

✓ Inventario

Controla las existencias, supervisa los movimientos de mercancía y garantiza la disponibilidad de productos.

✓ Marketing y servicio al cliente

Diseña campañas publicitarias, promociones y estrategias de fidelización para incrementar las ventas y fortalecer la imagen de la empresa. Atiende consultas, sugerencias y reclamos de los clientes, procurando una experiencia de compra satisfactoria.

✓ Operaciones

Supervisa el funcionamiento operativo de las sucursales, asegurando el cumplimiento de los procedimientos y estándares establecidos.

✓ Sucursales

Realizan la comercialización de los productos, administran su inventario local y brindan atención directa a los clientes.

Procesos del negocio

1. Propósito

El presente documento describe los procesos operativos de DataMart 360, detallando las actividades principales que permiten el funcionamiento diario de la empresa. Estos procesos servirán como base para el análisis de requerimientos, el diseño de la base de datos y el desarrollo de los sistemas de información.

2. Procesos Estratégicos

Son los procesos que permiten alcanzar los objetivos de la empresa.

- Gestión Comercial.
- Gestión de Compras.
- Gestión de Inventario.
- Gestión Logística.
- Gestión de Ventas.
- Atención al Cliente.
- Programa de Fidelización.

3. Procesos Operativos

3.1 Proceso de Compra de Mercancía

Objetivo

Abastecer el almacén central con productos provenientes de proveedores autorizados.

Responsable

Departamento de Compras.

Descripción

1. Se analiza el inventario del almacén central.
2. Se identifican los productos con bajo stock.
3. Se selecciona el proveedor correspondiente.
4. Se genera la orden de compra.
5. El proveedor despacha la mercancía hacia el almacén central.

3.2 Recepción e Inspección de Mercancía

Objetivo

Verificar que los productos recibidos cumplan con las condiciones pactadas antes de ingresar al inventario.

Responsable

Logística e Inventario.

Descripción

4. Se recibe la mercancía.
5. Se verifica la cantidad recibida.
6. Se inspecciona el estado físico de los productos.
7. Se valida la fecha de vencimiento cuando corresponda.
8. Si todo está correcto, la mercancía ingresa al inventario del almacén central.

3.3 Distribución hacia las Sucursales

Objetivo

Abastecer las sucursales según sus necesidades de inventario.

Responsable

Logística.

Descripción

1. La sucursal solicita productos al almacén central.
2. Se valida la disponibilidad.
3. Se prepara el pedido.
4. Se despacha la mercancía.
5. La sucursal confirma la recepción.
6. Se actualizan los inventarios de ambas ubicaciones.

3.4 Administración del Inventario

Objetivo

Mantener un control actualizado de las existencias en el almacén central y en cada sucursal.

Responsable

Inventario.

Descripción

1. Registrar entradas.
2. Registrar salidas.
3. Registrar transferencias.
4. Controlar stock mínimo.
5. Identificar productos próximos a agotarse.

3.5 Venta de Productos

Objetivo

Comercializar los productos disponibles a los clientes.

Responsable

Operaciones.

Descripción

1. El cliente selecciona los productos.
2. El cajero registra la venta.
3. Se aplican promociones si existen.
4. Se calcula el IVA correspondiente.
5. Se registra el método de pago.
6. Se genera la factura.
7. Se descuenta automáticamente el inventario.

3.6 Programa de Fidelización

Objetivo

Fidelizar a los clientes mediante la acumulación de puntos.

Responsable

Marketing.

Descripción

1. El cliente presenta su tarjeta.
2. El sistema calcula los puntos obtenidos.
3. Los puntos se acumulan.
4. Posteriormente pueden canjearse por descuentos.

3.7 Gestión de Promociones

Objetivo

Incrementar las ventas mediante campañas promocionales.

Responsable

Marketing.

Descripción

Las promociones podrán aplicarse a:

1. Productos individuales.
2. Categorías completas.

Cada producto únicamente podrá participar en una promoción activa al mismo tiempo.

3.8 Gestión de Clientes

Objetivo

Administrar la información de los clientes registrados.

Responsable

Servicio al Cliente.

Descripción

1. Registrar nuevos clientes.
2. Actualizar información.
3. Administrar la tarjeta de fidelización.
4. Consultar historial de compras.

3.9 Gestión de Empleados

Objetivo

Administrar la información del personal de la empresa.

Responsable

Recursos Humanos.

Descripción

1. Registrar empleados.
2. Asignar sucursal.

3. Asignar cargo.
4. Actualizar información laboral.

3.10 Relación entre procesos

Proceso	Depende de
Compra de Mercancía	Inventario
Recepción de Mercancía	Compra de Mercancía
Distribución	Recepción de Mercancía
Inventario	Distribución
Venta	Inventario
Facturación	Venta
Programa de Fidelización	Venta
Promociones	Marketing
Gestión de Clientes	Venta
Gestión de Empleados	Recursos Humanos

Reglas del negocio

Propósito

El presente documento establece las reglas del negocio que rigen el funcionamiento de DataMart 360. Estas reglas representan políticas y restricciones que deberán respetarse durante el diseño de la base de datos, el desarrollo del sistema y la operación diaria de la empresa.

Reglas del Negocio

Código	Categoría	Regla del negocio
RN-001	Ventas	Toda venta debe generar una factura.
RN-002	Ventas	No se permite eliminar una venta; únicamente podrá cambiar su estado cuando sea necesario.
RN-003	Ventas	Cada factura debe estar asociada a una única venta.
RN-004	Ventas	Una venta puede incluir uno o varios productos.
RN-005	Ventas	La cantidad máxima permitida por producto en una misma venta será de tres paquetes o su equivalente comercial.
RN-006	Ventas	Todas las ventas deben registrarse en el sistema.

Clientes

Código	Categoría	Regla del negocio
RN-007	Clientes	Un cliente puede registrarse una única vez en el sistema.
RN-008	Clientes	El registro del cliente permanecerá disponible para futuras compras.
RN-009	Clientes	Solo los clientes registrados podrán solicitar la tarjeta de fidelización.

Fidelización

Código	Categoría	Regla del negocio
RN-010	Fidelización	La tarjeta de puntos es opcional para los clientes.
RN-011	Fidelización	Los puntos se acumulan únicamente en compras asociadas a la tarjeta de fidelización.

Productos

Código	Categoría	Regla del negocio
RN-012	Productos	Cada producto pertenece únicamente a una categoría.
RN-013	Productos	Cada producto tendrá un proveedor principal asignado.
RN-014	Productos	Un proveedor puede suministrar múltiples productos.

Compras

Código	Categoría	Regla del negocio
RN-015	Compras	Solo el almacén central puede realizar compras a los proveedores.
RN-016	Compras	Ninguna sucursal puede comprar directamente a un proveedor.
RN-017	Compras	Toda mercancía recibida debe ser inspeccionada antes de ingresar al inventario.

Inventario

Código	Categoría	Regla del negocio
RN-018	Inventario	Cada sucursal administra su propio inventario.
RN-019	Inventario	El almacén central mantiene un inventario independiente del inventario de las sucursales.

RN-020	Inventario	Las sucursales únicamente pueden abastecerse desde el almacén central.
RN-021	Inventario	Toda transferencia entre el almacén central y una sucursal debe registrarse en el sistema.

Promociones

Código	Categoría	Regla del negocio
RN-022	Promociones	Las promociones pueden aplicarse a un producto o a una categoría.
RN-023	Promociones	Un producto no puede tener más de una promoción activa al mismo tiempo.

Empleados

Código	Categoría	Regla del negocio
RN-024	Empleados	Cada empleado pertenece únicamente a una sucursal.
RN-025	Empleados	Los empleados son registrados por el departamento de Recursos Humanos.

Seguridad

Código	Categoría	Regla del negocio
RN-026	Seguridad	Solo el administrador de la sucursal podrá modificar los precios de los productos.
RN-027	Seguridad	Solo el administrador de la sucursal podrá modificar el estado de una venta.

Horarios

Código	Categoría	Regla del negocio
RN-028	Operaciones	Todas las sucursales operan de lunes a sábado, de 09:00 a 18:00 horas.

Impuestos

Código	Categoría	Regla del negocio
RN-029	Facturación	El descuento se aplicará antes del cálculo del IVA cuando corresponda.
RN-030	Facturación	El IVA solo se aplicará a los productos gravados por la legislación vigente.

Observaciones

Las reglas del negocio podrán actualizarse conforme evolucionen los procesos de DataMart 360. Cualquier modificación deberá evaluarse antes de implementarse en la base de datos o en los sistemas de información para garantizar la consistencia del modelo de datos y el correcto funcionamiento de la operación.

Glosario del negocio

Propósito

El presente glosario reúne los principales términos utilizados en DataMart 360, proporcionando una definición clara y estandarizada para facilitar la comprensión del negocio y mantener un lenguaje común durante el desarrollo del proyecto.

Glosario de Términos

Término	Definición
Almacén Central	Instalación principal donde se reciben, inspeccionan, almacenan y distribuyen los productos hacia las sucursales.
Categoría	Clasificación utilizada para agrupar productos con características similares, como lácteos, bebidas o limpieza.
Cliente	Persona que adquiere productos en cualquiera de las sucursales de DataMart 360.
Compra	Proceso mediante el cual el almacén central adquiere mercancía de un proveedor autorizado.
Descuento	Reducción aplicada al precio de un producto como parte de una promoción o estrategia comercial.
Empleado	Persona contratada por DataMart 360 para desempeñar funciones dentro de una de sus áreas o sucursales.
Factura	Documento generado por el sistema que registra oficialmente una venta realizada a un cliente.
Fidelización	Estrategia orientada a incentivar la permanencia de los clientes mediante la acumulación de puntos y beneficios.
Inventario	Registro de las existencias disponibles de cada producto en el almacén central y en las sucursales.
IVA	Impuesto al Valor Agregado aplicado a determinados productos según la legislación vigente.

Logística	Área encargada de coordinar el almacenamiento, transporte y distribución de mercancías.
Método de Pago	Medio utilizado por el cliente para cancelar una compra, como efectivo, tarjeta o transferencia bancaria.
Producto	Artículo comercializado por DataMart 360 para satisfacer las necesidades de sus clientes.
Promoción	Estrategia comercial destinada a incentivar las ventas mediante descuentos, ofertas o beneficios especiales.
Proveedor	Empresa o persona autorizada para suministrar productos al almacén central de DataMart 360.
Punto de Fidelización	Unidad acumulada por los clientes afiliados al programa de fidelización, que puede utilizarse para obtener beneficios o descuentos.
Reposición	Proceso mediante el cual el almacén central abastece nuevamente a una sucursal cuando su inventario disminuye.
Sucursal	Establecimiento físico donde se comercializan los productos directamente a los clientes.
Factura	Comprobante entregado al cliente con el detalle de la venta realizada. En este proyecto será equivalente a la factura simplificada.
Transferencia de Inventario	Movimiento de mercancía desde el almacén central hacia una sucursal.
Venta	Transacción mediante la cual un cliente adquiere uno o varios productos disponibles en una sucursal.

Abreviaturas

Abreviatura	Significado
TI	Tecnología de la Información
IVA	Impuesto al Valor Agregado
KPI	Indicador Clave de Desempeño (<i>Key Performance Indicator</i>)

SKU	Código único utilizado para identificar un producto dentro del inventario.
-----	--

Convenciones del Proyecto

Para mantener consistencia durante el desarrollo del proyecto, se adoptan las siguientes convenciones:

- El término **almacén central** se utilizará para referirse al centro de distribución principal de la empresa.
- El término **sucursal** hará referencia a cualquier establecimiento de venta al público.
- El término **cliente** identificará tanto a clientes ocasionales como a clientes afiliados al programa de fidelización.
- El término **producto** representará cualquier artículo comercializado por la empresa, independientemente de su categoría.
- El término **venta** corresponderá a una transacción completa registrada mediante una factura.

Indicadores Clave de Desempeño (KPIs)

Propósito

El presente documento define los Indicadores Clave de Desempeño (KPIs) que permitirán evaluar el rendimiento operativo y comercial de DataMart 360. Estos indicadores servirán como base para el desarrollo de los dashboards en Power BI y facilitarán la toma de decisiones basada en datos.

Indicadores de Ventas

KPI	Descripción	Fórmula	Frecuencia	Responsable
Ventas Totales	Valor total vendido.	SUM(Ventas)	Diario/Mensual	Finanzas
Número de Ventas	Cantidad de facturas emitidas.	COUNT(Facturas)	Diario	Operaciones
Ticket Promedio	Valor promedio por compra.	$\frac{\text{Ventas Totales}}{\text{Número de Ventas}}$	Diario	Finanzas
Productos Vendidos	Total de unidades vendidas.	SUM(Cantidad)	Diario	Operaciones

Indicadores de Inventario

KPI	Descripción	Frecuencia	Responsable
Productos con Bajo Stock	Productos por debajo del stock mínimo.	Diario	Inventario
Rotación de Inventario	Velocidad con que se venden los productos.	Mensual	Inventario
Valor del Inventario	Valor económico del inventario disponible.	Diario	Finanzas
Transferencias Realizadas	Cantidad de abastecimientos enviados a sucursales.	Mensual	Logística

Indicadores Comerciales

KPI		Descripción	Frecuencia	Responsable
Productos Vendidos	Más	Ranking de productos.	Mensual	Marketing
Categorías Vendidas	Más	Ranking de categorías.	Mensual	Marketing
Promociones Exitosas	Más	Promociones con mayor impacto en ventas.	Mensual	Marketing
Clientes Frecuentes		Clientes con mayor número de compras.	Mensual	Marketing

Indicadores Financieros

KPI	Descripción
Ingresos Totales	
Ventas por Sucursal	
Ventas por Ciudad	
Ventas por Método de Pago	
Descuento Total Aplicado	
IVA Recaudado	

Indicadores de Fidelización

KPI	Descripción
Clientes Registrados	
Clientes con Tarjeta	
Puntos Acumulados	
Puntos Canjeados	
Compras de Clientes Fidelizados	

Visualizaciones sugeridas

- Tarjetas KPI
- Gráficos de barras
- Gráficos de líneas
- Mapas por ciudad
- Matrices
- Árbol de descomposición
- Segmentadores
- Indicadores de tendencia

Diccionario del Negocio

Este documento será el puente entre el análisis del negocio y el modelo de datos.

Son entidades del negocio.

Entidad	Descripción
Cliente	Persona que realiza compras en la empresa.
Empleado	Persona contratada por la empresa.
Producto	Artículo comercializado por la empresa.
Categoría	Clasificación de productos.
Proveedor	Empresa que suministra productos.
Compra	Adquisición de mercancía al proveedor.
Detalle de Compra	Productos incluidos en una compra.
Inventario	Existencias disponibles.
Transferencia	Movimiento de productos desde la central hacia una sucursal.
Sucursal	Punto de venta de la empresa.
Factura	Documento generado por una venta.
Detalle de Factura	Productos vendidos en una factura.
Promoción	Campaña comercial aplicada a productos o categorías.
Tarjeta de Fidelización	Identificador del programa de puntos.
Puntos	Beneficios acumulados por los clientes.
Método de Pago	Forma de pago utilizada en una compra.
Usuario del Sistema	Persona autorizada para utilizar el sistema.

Al finalizar este documento tendremos prácticamente definidas las futuras tablas de PostgreSQL.

Catálogo Maestro

Este documento reúne todos los valores controlados del sistema. Más adelante, muchos de ellos se convertirán en tablas catálogo.

Categorías de Productos

- Lácteos
- Carnes
- Embutidos
- Panadería
- Frutas
- Verduras
- Bebidas
- Snacks
- Limpieza
- Cuidado Personal
- Hogar
- Mascotas
- Congelados

Métodos de Pago

- Efectivo
- Tarjeta
- Transferencia
- Billetera electrónica / Pago móvil

Tipos de Promoción

- 2x1
- Descuento por porcentaje.
- Descuento por monto fijo.
- Promoción por temporada.
- Productos en liquidación.
- Combos.

Estados de Factura

- Emitida
- Anulada
- corregida

Estados del Inventario

- Disponible
- Bajo Stock
- Agotado

Tipos de Cliente

- Cliente ocasional
- Cliente registrado
- Cliente fidelizado

Tipos de Documento

- Cédula
- Pasaporte
- RUC

Roles del Sistema

- Administrador General
- Administrador de Sucursal
- Cajero
- Inventario
- Recursos Humanos
- Compras
- Marketing

Matriz de Requerimientos Funcionales y No Funcionales

Este documento servirá de base para el diseño del sistema.

Requerimientos Funcionales

Código	Requerimiento
RF-001	Registrar clientes.
RF-002	Registrar empleados.
RF-003	Registrar proveedores.
RF-004	Registrar productos.
RF-005	Registrar categorías.
RF-006	Registrar compras.
RF-007	Registrar recepción de mercancía.
RF-008	Registrar transferencias a sucursales.
RF-009	Registrar ventas.
RF-010	Emitir facturas.
RF-011	Aplicar promociones.
RF-012	Administrar inventario.
RF-013	Administrar puntos de fidelización.
RF-014	Generar reportes.
RF-015	Consultar indicadores de negocio.

Requerimientos No Funcionales

Código	Requerimiento
RNF-001	El sistema deberá garantizar la integridad de la información.
RNF-002	Toda la información deberá almacenarse en PostgreSQL.
RNF-003	El acceso estará controlado mediante usuarios y roles.
RNF-004	Todas las operaciones deberán quedar registradas.
RNF-005	El sistema deberá mantener la disponibilidad de la información.
RNF-006	Solo usuarios autorizados podrán modificar información crítica.
RNF-007	La base de datos deberá encontrarse normalizada hasta Tercera Forma Normal (3FN).
RNF-008	La información deberá ser compatible con herramientas de análisis como Excel y Power BI.

Casos de uso del negocio

No son diagramas UML todavía. Son descripciones de las acciones que realizan los usuarios del negocio.

- Registrar un cliente.
- Registrar un producto.
- Registrar un proveedor.
- Realizar una compra.
- Recepcionar mercancía.
- Transferir productos a una sucursal.
- Registrar una venta.
- Emitir una factura.
- Aplicar una promoción.
- Consultar el inventario.
- Consultar reportes.
- Administrar empleados.
- Administrar usuarios del sistema.

Más adelante podremos convertir estos casos de uso en diagramas UML si lo deseas. Tenerlos documentados desde ahora hará que la transición al diseño de la base de datos y, eventualmente, al desarrollo de una aplicación, sea mucho más natural.

MÓDULO 2

Diseño de Base de Datos

Definición de la nomenclatura

- Convención para nombres de tablas: plural
- Convención para nombres de columnas: snake_case
- Tipo de claves primarias (PK): BIGSERIAL
- Convención para claves foráneas (FK): id_ seguido del nombre de la tabla padre en singular.

Tipos de datos por defecto

Para texto:

- Usaremos VARCHAR(N) para campos con límites lógicos claros (ej: codigo_barras VARCHAR(20), nit_empresa VARCHAR(50)) y
- TEXT solo para descripciones muy largas o notas.

Para dinero/precios

- Usaremos estrictamente NUMERIC (12, 2) o DECIMAL (12, 2)

Para fechas y zonas horarias

- TIMESTAMPTZ

Reglas de nomenclaturas para restricciones e índices

Tipo de Restricción / Índice	Prefijo	Estructura del Nombre	Ejemplo Práctico
Clave Primaria (Primary Key)	pk_	pk_[nombre_tabla]	pk_productos
Clave Foránea (Foreign Key)	fk_	fk_[tabla_origen]_[tabla_destino]	fk_detalle_venta s_productos
Restricción Única (Unique)	uq_	uq_[nombre_tabla]_[columna]	uq_clientes_documento_identidad

Validación Chequeo (Check)	/	ck_	ck_[nombre_tabla] _[columna]	ck_productos_prec io_positivo
Índice (Index)	Estándar	idx_	idx_[nombre_tabla] _[columna]	idx_ventas_fecha_ venta

Lista oficial de entidades

1. Core de Ventas y Facturación	
clientes	Almacena los datos maestros de las personas que realizan compras (Nombres, RUC/Cédula, contacto).
metodos_pago	Catálogo de las formas de pago autorizadas en las cajas (Efectivo, Tarjeta de Crédito, Débito, etc.).
facturas	Cabecera transaccional que registra la información general de una venta (Fecha, cliente, sucursal, método de pago y totales históricos).
detalles_facturas	Desglose o líneas de cada factura. Registra los productos vendidos, cantidades exactas (con decimales para peso) y precios/impuestos históricos.
2. Gestión de Productos e Inventarios	
categorias	Clasificación base de los productos para ordenamiento en pasillos y reportes (ej
productos	Maestro central de todos los artículos del supermercado, incluyendo su código de barras, descripción y precio de venta actual.
sucursales	Registro de los puntos de venta físicos de la cadena y de la bodega central o centro de distribución.
inventarios	Tabla intermedia que controla el stock físico y disponible de cada producto específico por cada sucursal individual.

despachos_sucursales	Cabecera logística que documenta el traslado interno de mercancía en camiones desde la bodega central hacia una sucursal destino.
detalles_despachos	Detalle de los productos enviados en un despacho, permitiendo comparar la cantidad enviada contra la recibida para auditar mermas.
3. Abastecimiento y Proveedores (Compras)	
proveedores	Maestro de empresas externas que suministran los productos al supermercado (Razón social, identificación fiscal, contacto).
compras	Cabecera que ampara la adquisición de mercancía a un proveedor, vinculada a la sucursal donde se recibe y a la factura física del proveedor.
detalles_compras	Desglose de la compra que registra qué productos ingresaron al inventario y el costo unitario exacto cobrado por el proveedor.
4. Fidelización y Marketing	
promociones	Registro de campañas comerciales y descuentos (temporales, porcentuales o fijos) aplicados a las ventas.
tarjetas_fidelizacion	Identificador del programa de lealtad de la cadena. Almacena de forma directa el saldo acumulado de puntos del cliente afiliado.
Productos_promociones	Tabla de rompimiento que conecta los productos con las promociones que tienen activas.
5. Gestión de Accesos, Personal y Seguridad (Auditoría)	
empleados	Datos del personal contratado por la empresa (cargos, datos de contacto, fecha de ingreso).
usuarios_sistema	Credenciales de acceso al software vinculadas a los empleados, con contraseñas encriptadas y roles específicos (Cajero, Admin).

auditorias_sistema	Bitácora técnica centralizada (log) que registra qué usuario hizo qué acción crítica (ej. cambiar un precio o eliminar lógicamente), en qué tabla y en qué momento exacto.
---------------------------	--

NOTA TÉCNICA: Resumen de Normalización (DataMart 360)

El objetivo de la normalización es eliminar la redundancia de datos, evitar anomalías en las actualizaciones y garantizar la integridad referencial antes de la implementación en PostgreSQL.

- **1FN (Primera Forma Normal) - "Atomicidad":**
 - *Regla:* Cada celda debe contener un único valor (atómico), no se permiten listas ni campos multivalor, y no debe haber filas repetidas.
 - *En DataMart 360:* Separamos los datos generales de la venta (Cabecera en `facturas`) de los artículos comprados (Líneas en `detalles_facturas`).
- **2FN (Segunda Forma Normal) - "Dependencia de la Clave":**
 - *Regla:* Debe cumplir la 1FN y todos los atributos que no son clave deben depender por completo de la Clave Primaria (PK) de esa tabla, no de una parte de ella.
 - *En DataMart 360:* Extrajimos los datos fijos del artículo (`nombre`, `codigo_barras`) del detalle de la venta hacia su propia tabla maestra (`productos`), dejando solo el `id_producto` como referencia.
- **3FN (Tercera Forma Normal) - "Independencia Transitiva":**
 - *Regla:* Debe cumplir la 2FN y ningún atributo debe depender de otro atributo que no sea la Clave Primaria (evitar relaciones indirectas del tipo A – B – C)
 - *En DataMart 360:* Separamos las descripciones de las categorías a su propia tabla (`categorias`) y los datos del cliente a `clientes`, evitando que la tabla `productos` o `facturas` carguen con textos repetidos e innecesarios.

Conclusión: El diseño actual de DataMart 360 se encuentra en **Tercera Forma Normal (3FN)**, garantizando que cada dato se almacene en un único lugar lógico en el tiempo.

Catálogo de Restricciones lógicas (Constraints)

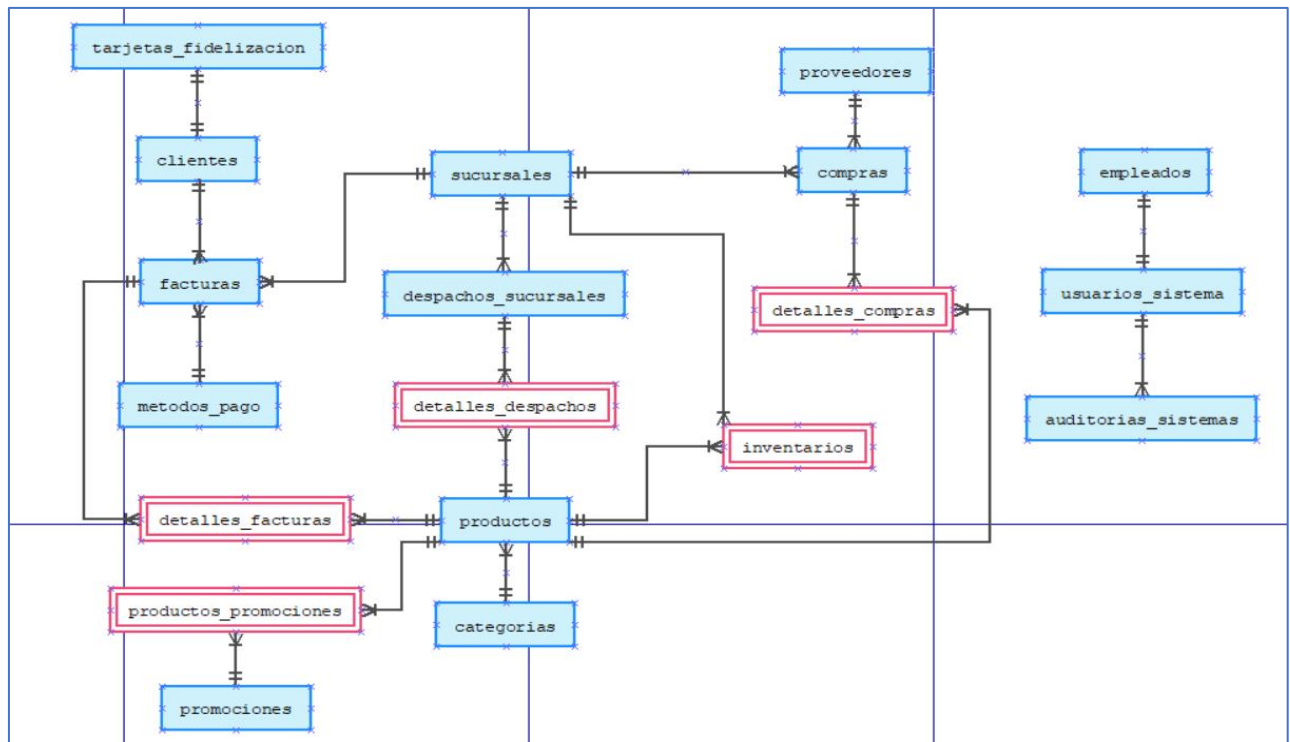
1. Restricciones de Unicidad (uq_) - Evitan duplicados ilegales

- `uq_clientes_identificacion`: El número de cédula o RUC en la tabla `clientes` no se puede repetir.
- `uq_productos_codigo_barras`: El código de barras en `productos` debe ser único en todo el supermercado.
- `uq_facturas_numero`: El número secuencial legal de la factura no se puede duplicar.
- `uq_usuarios_username`: Dos empleados no pueden tener el mismo nombre de usuario (ej. no puede haber dos `maria.lopez`).

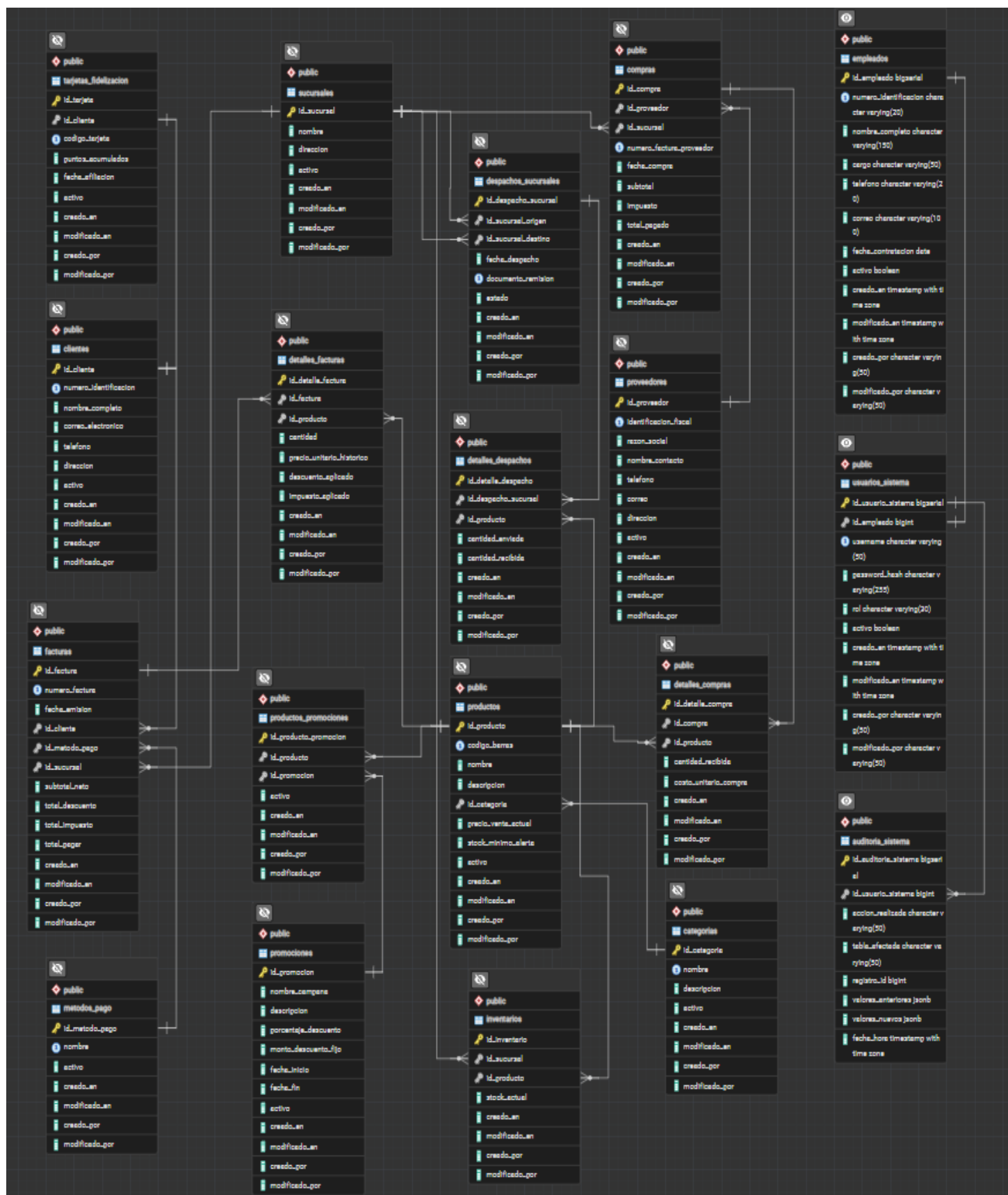
2. Restricciones de Validación (ck_) - Evitan datos matemáticamente ilógicos

- `ck_productos_precio_positivo`: En `productos`, `precio_venta_actual` > 0. (No regalamos productos).
- `ck_inventarios_stock_no_negativo`: En `inventarios`, `stock_actual` >= 0. (No puede haber stock físico negativo en una tienda).
- `ck_detalle_facturas_cantidad`: En `detalle_facturas`, `cantidad` > 0. (Nadie compra cero o menos un producto).
- `ck_compras_costo_pos`: En `detalle_compras`, `costo_unitario_compra` > 0. (Toda mercancía que compramos tiene un costo).
- `ck_promo_porcentaje`: En `promociones`, `porcentaje_descuento` BETWEEN 0 AND 100. (Un descuento no puede ser menor al 0% ni mayor al 100%).
- `ck_promo_fechas`: En `promociones`, `fecha_fin` > `fecha_inicio`. (Una campaña no puede terminar antes de haber empezado).

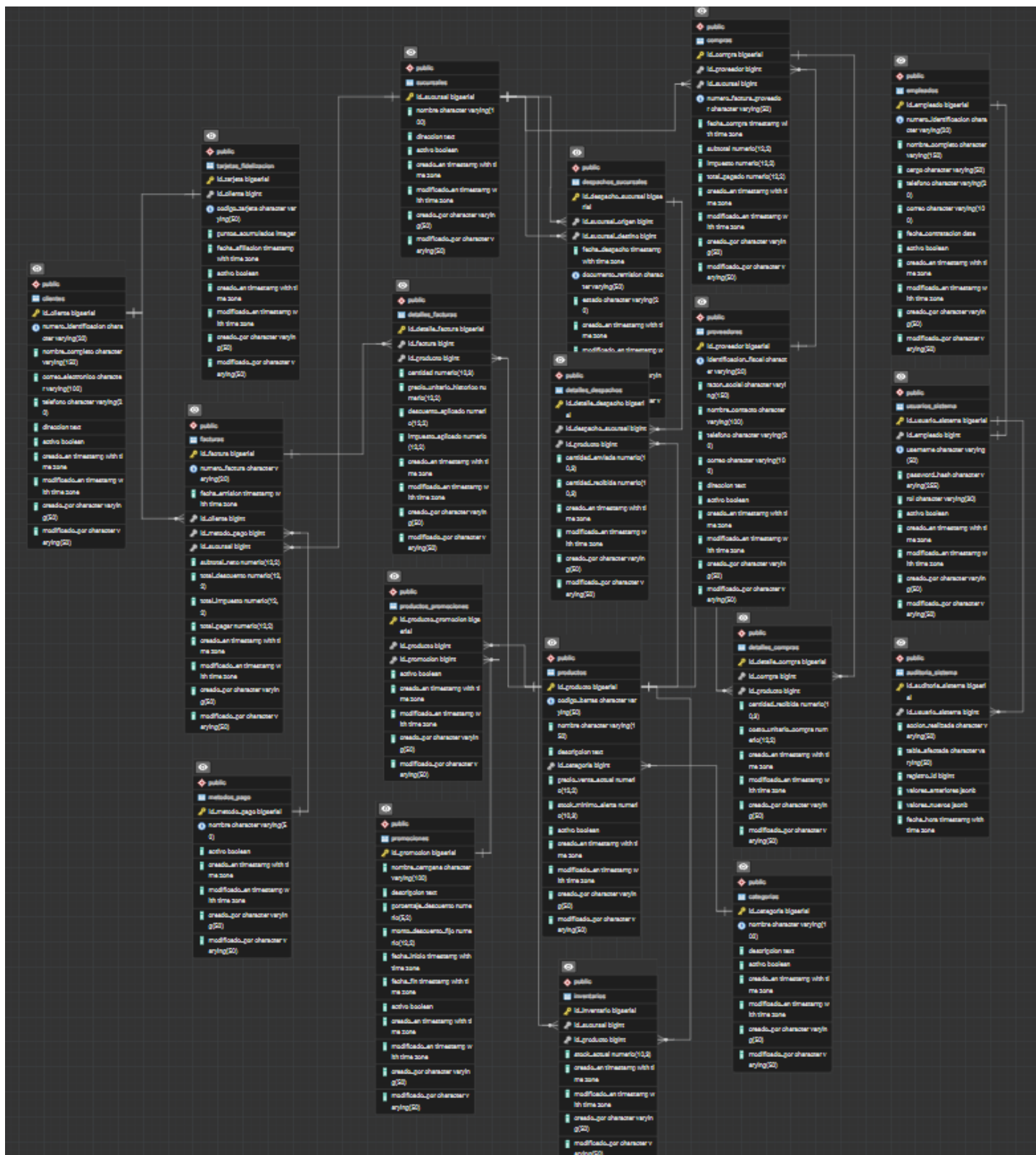
MODELO CONCEPTUAL



MODELO LÓGICO



MODELO RELACIONAL



MÓDULO 3

Implementación PostgreSQL

Script SQL completo.

```
-- Database: datamart360

-- DROP DATABASE IF EXISTS datamart360;

CREATE DATABASE datamart360
    WITH
        OWNER = postgres
        ENCODING = 'UTF8'
        LC_COLLATE = 'Spanish_Ecuador.1252'
        LC_CTYPE = 'Spanish_Ecuador.1252'
        LOCALE_PROVIDER = 'libc'
        TABLESPACE = pg_default
        CONNECTION LIMIT = -1
        IS_TEMPLATE = False;

--CREACION DE TABLAS

--TABLA CLIENTES
CREATE TABLE clientes (
    id_cliente BIGSERIAL,
    numero_identificacion VARCHAR(20),
    nombre_completo VARCHAR(150) NOT NULL,
    correo_electronico VARCHAR(100),
    telefono VARCHAR(20),
    direccion TEXT,

    -- Campos de Auditoría y Estado
    activo BOOLEAN DEFAULT TRUE NOT NULL,
    creado_en TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP NOT NULL,
    modificado_en TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP NOT NULL,
    creado_por VARCHAR(50) DEFAULT 'system' NOT NULL,
    modificado_por VARCHAR(50),

    -- Restricciones (Constraints)
    CONSTRAINT pk_clientes PRIMARY KEY (id_cliente),
    CONSTRAINT uq_clientes_identificacion UNIQUE (numero_identificacion)
);
```

```
--TABLA METODOS_PAGO
CREATE TABLE metodos_pago (
    id_metodo_pago BIGSERIAL,
    nombre VARCHAR(50) NOT NULL,

    -- Campos de auditoría y estado
    activo BOOLEAN DEFAULT TRUE NOT NULL,
    creado_en TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP NOT NULL,
    modificado_en TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP NOT NULL,
    creado_por VARCHAR(50) DEFAULT 'system' NOT NULL,
    modificado_por VARCHAR(50),

    -- Restricciones (Constraints)
    CONSTRAINT pk_metodos_pago PRIMARY KEY (id_metodo_pago),
    CONSTRAINT uq_metodos_pago_nombre UNIQUE (nombre)
);

--TABLA SUCURSALES
CREATE TABLE sucursales(
    id_sucursal BIGSERIAL,
    nombre VARCHAR(100) NOT NULL,
    direccion TEXT NOT NULL,

    --Campos de auditoria y estado
    activo BOOLEAN DEFAULT TRUE NOT NULL,
    creado_en TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP NOT NULL,
    modificado_en TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP NOT NULL,
    creado_por VARCHAR(50) DEFAULT 'system' NOT NULL,
    modificado_por VARCHAR(50),

    --Restriccion
    CONSTRAINT pk_sucursales PRIMARY KEY (id_sucursal)
);

--TABLA CATEGORIAS
CREATE TABLE categorias(
    id_categoria BIGSERIAL,
    nombre VARCHAR(100) NOT NULL,
    descripcion TEXT,

    --Campos de auditoria y estado
    activo BOOLEAN DEFAULT TRUE NOT NULL,
    creado_en TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP NOT NULL,
    modificado_en TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP NOT NULL,
    creado_por VARCHAR(50) DEFAULT 'system' NOT NULL,
    modificado_por VARCHAR(50),

    --Restricciones
```

```

        CONSTRAINT pk_categorias PRIMARY KEY (id_categoria),
        CONSTRAINT uq_nombre_categorias UNIQUE (nombre)
    );

--TABLA PROVEEDORES
CREATE TABLE proveedores(
    id_proveedor BIGSERIAL,
    identificacion_fiscal VARCHAR(20) NOT NULL,
    razon_social VARCHAR(150) NOT NULL,
    nombre_contacto VARCHAR(100),
    telefono VARCHAR(20),
    correo VARCHAR(100),
    direccion TEXT,

    --Campos de auditoria y estado
    activo BOOLEAN DEFAULT TRUE NOT NULL,
    creado_en TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP NOT NULL,
    modificado_en TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP NOT NULL,
    creado_por VARCHAR(50) DEFAULT 'system' NOT NULL,
    modificado_por VARCHAR(50),

    --Restricciones
    CONSTRAINT pk_proveedores PRIMARY KEY (id_proveedor),
    CONSTRAINT uq_proveedores_fiscal UNIQUE (identificacion_fiscal)
);

--TABLA PROMOCIONES
CREATE TABLE promociones (
    id_promocion BIGSERIAL,
    nombre_campana VARCHAR(100) NOT NULL,
    descripcion TEXT,
    porcentaje_descuento NUMERIC(5,2) DEFAULT 0.00,
    monto_descuento_fijo NUMERIC(12,2) DEFAULT 0.00,
    fecha_inicio TIMESTAMPTZ NOT NULL,
    fecha_fin TIMESTAMPTZ NOT NULL,

    -- Campos de auditoría y estado
    activo BOOLEAN DEFAULT TRUE NOT NULL,
    creado_en TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP NOT NULL,
    modificado_en TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP NOT NULL,
    creado_por VARCHAR(50) DEFAULT 'system' NOT NULL,
    modificado_por VARCHAR(50),

    -- Restricciones (Constraints)
    CONSTRAINT pk_promociones PRIMARY KEY (id_promocion),
    CONSTRAINT ck_promo_porcentaje CHECK (porcentaje_descuento BETWEEN 0.00
AND 100.00),
    CONSTRAINT ck_promo_fechas CHECK (fecha_fin > fecha_inicio)
);

```

```

--TABLA EMPLEADOS
CREATE TABLE empleados(
    id_empleado BIGSERIAL,
    numero_identificacion VARCHAR(20) NOT NULL,
    nombre_completo VARCHAR(150) NOT NULL,
    cargo VARCHAR(50) NOT NULL,
    telefono VARCHAR(20),
    correo VARCHAR(100),
    fecha_contratacion DATE NOT NULL,

    --Campos de auditoria y estado
    activo BOOLEAN DEFAULT TRUE NOT NULL,
    creado_en TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP NOT NULL,
    modificado_en TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP NOT NULL,
    creado_por VARCHAR(50) DEFAULT 'system' NOT NULL,
    modificado_por VARCHAR(50),

    --Restricciones
    CONSTRAINT pk_empleados PRIMARY KEY (id_empleado),
    CONSTRAINT uq_empleados_identificacion UNIQUE (numero_identificacion)
);

--TABLA PRODUCTOS
CREATE TABLE productos (
    id_producto BIGSERIAL,
    codigo_barras VARCHAR(50) NOT NULL,
    nombre VARCHAR(150) NOT NULL,
    descripcion TEXT,
    id_categoria BIGINT NOT NULL, -- <--- Esta columna guardará el ID de la categoría
    precio_venta_actual NUMERIC(12,2) NOT NULL,
    stock_minimo_alerta NUMERIC(10,3) DEFAULT 0.000,

    -- Campos de auditoría y estado
    activo BOOLEAN DEFAULT TRUE NOT NULL,
    creado_en TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP NOT NULL,
    modificado_en TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP NOT NULL,
    creado_por VARCHAR(50) DEFAULT 'system' NOT NULL,
    modificado_por VARCHAR(50),

    -- Restricciones (Constraints)
    CONSTRAINT pk_productos PRIMARY KEY (id_producto),
    CONSTRAINT uq_productos_codigo_barras UNIQUE (codigo_barras),

    -- ¡La regla de oro del precio positivo!
    CONSTRAINT ck_productos_precio_positivo CHECK (precio_venta_actual > 0),

    -- ¡Aquí hacemos la magia de la conexión física!

```

```

        CONSTRAINT fk_productos_categorias FOREIGN KEY (id_categoria) REFERENCES
categorias(id_categoria)
    );

--TABLA USUARIOS_SISTEMA
CREATE TABLE usuarios_sistema (
    id_usuario_sistema BIGSERIAL,
    id_empleado BIGINT NOT NULL,
    username VARCHAR(50) NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    rol VARCHAR(30) NOT NULL,

    -- Campos de auditoría y estado
    activo BOOLEAN DEFAULT TRUE NOT NULL,
    creado_en TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP NOT NULL,
    modificado_en TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP NOT NULL,
    creado_por VARCHAR(50) DEFAULT 'system' NOT NULL,
    modificado_por VARCHAR(50),

    -- Restricciones (Constraints)
    CONSTRAINT pk_usuarios_sistema PRIMARY KEY (id_usuario_sistema),
    CONSTRAINT uq_usuario_empleado UNIQUE (id_empleado), -- Garantiza relación
1 a 1 (un empleado, un usuario)
    CONSTRAINT fk_usuarios_empleados FOREIGN KEY (id_empleado) REFERENCES
empleados(id_empleado), -- Conecta con empleados
    CONSTRAINT uq_usuarios_username UNIQUE (username) -- Evita nombres de
usuario duplicados
);

--TABLA TARJETAS_FIDELIZACION
CREATE TABLE tarjetas_fidelizacion(
    id_tarjeta BIGSERIAL,
    id_cliente BIGINT NOT NULL,
    codigo_tarjeta VARCHAR(50) NOT NULL,
    puntos_acumulados INT NOT NULL DEFAULT 0,
    fecha_afiliacion TIMESTAMPTZ NOT NULL,

    --Campos de auditoria y estado
    activo BOOLEAN DEFAULT TRUE NOT NULL,
    creado_en TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP NOT NULL,
    modificado_en TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP NOT NULL,
    creado_por VARCHAR(50) DEFAULT 'system' NOT NULL,
    modificado_por VARCHAR(50),

    --Restricciones
    CONSTRAINT pk_tarjetas_fidelizacion PRIMARY KEY (id_tarjeta),
    CONSTRAINT fk_tarjetas_fidelizacion_cliente FOREIGN KEY (id_cliente)
REFERENCES clientes(id_cliente),
    CONSTRAINT uq_tarjeta_cliente UNIQUE (id_cliente),

```

```

        CONSTRAINT uq_codigo_tarjeta UNIQUE (codigo_tarjeta),
        CONSTRAINT ck_puntos_no_negativos CHECK (puntos_acumulados >=0)

    );

--TABLA FACTURAS cabecera
    CREATE TABLE facturas(
        id_factura BIGSERIAL,
        numero_factura VARCHAR(20) NOT NULL,
        fecha_emision TIMESTAMPTZ NOT NULL,
        id_cliente BIGINT,
        id_metodo_pago BIGINT,
        id_sucursal BIGINT,

        --Totales base
        subtotal_netto NUMERIC(12,2) NOT NULL,
        total_descuento NUMERIC(12,2) NOT NULL,
        total_impuesto NUMERIC(12,2) NOT NULL,
        total_pagar NUMERIC(12,2) NOT NULL,

        --Campos de auditoria y estado
        creado_en TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP NOT NULL,
        modificado_en TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP NOT NULL,
        creado_por VARCHAR(50) DEFAULT 'system' NOT NULL,
        modificado_por VARCHAR(50),

        --Restricciones
        CONSTRAINT pk_facturas PRIMARY KEY (id_factura),
        CONSTRAINT uq_facturas_numero UNIQUE (numero_factura),
        CONSTRAINT fk_facturas_clientes FOREIGN KEY (id_cliente) REFERENCES
clientes(id_cliente),
        CONSTRAINT fk_facturas_metodos_pago FOREIGN KEY (id_metodo_pago)
REFERENCES metodos_pago (id_metodo_pago),
        CONSTRAINT fk_facturas_sucursal FOREIGN KEY (id_sucursal) REFERENCES
sucursales(id_sucursal),

        -- Validaciones de seguridad (Evitamos montos negativos y números de
factura vacíos)
        CONSTRAINT ck_factura_valores_positivos CHECK (subtotal_netto >= 0 AND
total_descuento >= 0 AND total_impuesto >= 0 AND total_pagar >= 0),
        CONSTRAINT ck_numero_factura_no_vacio CHECK (LENGTH(TRIM(numero_factura))
> 0)
    );

--TABLA COMPRAS
    CREATE TABLE compras(
        id_compra BIGSERIAL,
        id_proveedor BIGINT NOT NULL,
        id_sucursal BIGINT NOT NULL,

```

```

numero_factura_proveedor VARCHAR(50) NOT NULL,
fecha_compra TIMESTAMPTZ NOT NULL,

--Totales base
subtotal NUMERIC(12,2) NOT NULL,
impuesto NUMERIC(12,2) NOT NULL,
total_pagado NUMERIC(12,2) NOT NULL,

--Campos de auditoria y estado
creado_en TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP NOT NULL,
modificado_en TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP NOT NULL,
creado_por VARCHAR(50) DEFAULT 'system' NOT NULL,
modificado_por VARCHAR(50),

--Restricciones
CONSTRAINT pk_compras PRIMARY KEY (id_compra),
CONSTRAINT fk_compras_proveedores FOREIGN KEY (id_proveedor) REFERENCES
proveedores(id_proveedor),
CONSTRAINT fk_compras_sucursales FOREIGN KEY (id_sucursal) REFERENCES
sucursales(id_sucursal)

-- Validaciones lógicas (Checks)
CONSTRAINT ck_compras_valores_positivos CHECK (subtotal >= 0 AND impuesto
>= 0 AND total_pagado >= 0),
CONSTRAINT ck_num_factura_prov_no_vacio CHECK
(LENGTH(TRIM(numero_factura_proveedor)) > 0),

-- Evita registrar la misma factura del mismo proveedor dos veces (control de
duplicados)
CONSTRAINT uq_proveedor_factura UNIQUE (id_proveedor,
numero_factura_proveedor)
);

--TABLA DESPACHO_SUCURSALES
CREATE TABLE despachos_sucursales (
id_despacho_sucursal BIGSERIAL,
id_sucursal_origen BIGINT NOT NULL,
id_sucursal_destino BIGINT NOT NULL,
fecha_despacho TIMESTAMPTZ NOT NULL,
documento_remision VARCHAR(50),
estado VARCHAR(20) NOT NULL, -- Lo hacemos obligatorio para controlar el
flujo

-- Campos de auditoría y estado
creado_en TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP NOT NULL,
modificado_en TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP NOT NULL,
creado_por VARCHAR(50) DEFAULT 'system' NOT NULL,
modificado_por VARCHAR(50),

```

```

-- Restricciones (Constraints)
    CONSTRAINT pk_despachos_sucursales PRIMARY KEY (id_despacho_sucursal), -
- Sintaxis corregida

    -- Ambas llaves foráneas apuntan correctamente a la misma tabla sucursales.
    ¡Brillante!
    CONSTRAINT fk_despachos_origen FOREIGN KEY (id_sucursal_origen) REFERENCES
sucursales(id_sucursal),
    CONSTRAINT fk_despachos_destino FOREIGN KEY (id_sucursal_destino)
REFERENCES sucursales(id_sucursal), -- Corregido el punto por coma

    CONSTRAINT uq_despachos_remision UNIQUE (documento_remision),

    -- ¡Regla de oro de la logística! Evita despacharse a la misma sucursal
    CONSTRAINT ck_despachos_origen_destino_distintos CHECK (id_sucursal_origen
<> id_sucursal_destino),

    -- Controlamos que el estado solo sea uno de los permitidos por tu negocio
    CONSTRAINT ck_despachos_estado_valido CHECK (estado IN ('Borrador', 'En
Transito', 'Entregado', 'Anulado'))
);

--TABLA DETALLES_FACTURAS
CREATE TABLE detalles_facturas(
    id_detalle_factura BIGSERIAL,
    id_factura BIGINT NOT NULL,
    id_producto BIGINT NOT NULL,
    cantidad NUMERIC(10,3) NOT NULL,
    precio_unitario_historico NUMERIC(12,2) NOT NULL,
    descuento_aplicado NUMERIC(12,2) DEFAULT 0.00,
    impuesto_aplicado NUMERIC(12,2) NOT NULL,

    -- Campos de auditoría y estado
    creado_en TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP NOT NULL,
    modificado_en TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP NOT NULL,
    creado_por VARCHAR(50) DEFAULT 'system' NOT NULL,
    modificado_por VARCHAR(50),

    --Restricciones
    CONSTRAINT pk_detalle_facturas PRIMARY KEY (id_detalle_factura),
    CONSTRAINT fk_detalle_facturas_facturas FOREIGN KEY (id_factura)
REFERENCES facturas(id_factura),
    CONSTRAINT fk_detalle_facturas_productos FOREIGN KEY (id_producto)
REFERENCES productos(id_producto),

    -- Validaciones de seguridad para evitar errores de digitación o bugs
    CONSTRAINT ck_detalle_facturas_valores_positivos CHECK (cantidad > 0 AND
precio_unitario_historico >= 0 AND descuento_aplicado >= 0 AND impuesto_aplicado
>= 0)

```

```

);

--TABLA INVENTARIOS
CREATE TABLE inventarios(
    id_inventario BIGSERIAL,
    id_sucursal BIGINT NOT NULL,
    id_producto BIGINT NOT NULL,
    stock_actual NUMERIC(10,3) DEFAULT 0.000 NOT NULL,

    --Campos de auditoria y estado
    creado_en TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP NOT NULL,
    modificado_en TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP NOT NULL,
    creado_por VARCHAR(50) DEFAULT 'system' NOT NULL,
    modificado_por VARCHAR (50),

    --Restricciones
    CONSTRAINT pk_inventarios PRIMARY KEY (id_inventario),
    CONSTRAINT fk_inventarios_sucursales FOREIGN KEY (id_sucursal) REFERENCES
sucursales (id_sucursal),
    CONSTRAINT fk_inventarios_productos FOREIGN KEY (id_producto) REFERENCES
productos (id_producto),
    CONSTRAINT uq_sucursal_productos UNIQUE (id_sucursal, id_producto),
    CONSTRAINT ck_inventarios_stock_no_negativo CHECK (stock_actual >=0)

);

--TABLA DETALLES_DESPACHOS
CREATE TABLE detalles_despachos (
    id_detalle_despacho BIGSERIAL,
    id_despacho_sucursal BIGINT NOT NULL,
    id_producto BIGINT NOT NULL,
    cantidad_enviada NUMERIC(10,3) NOT NULL,
    cantidad_recibida NUMERIC(10,3),

    -- Campos de auditoría
    creado_en TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP NOT NULL,
    modificado_en TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP NOT NULL,
    creado_por VARCHAR(50) DEFAULT 'system' NOT NULL, -- Corregido de
creador_por
    modificado_por VARCHAR(50),

    -- Restricciones (Constraints)
    CONSTRAINT pk_detalle_despachos PRIMARY KEY (id_detalle_despacho),
    CONSTRAINT fk_detalle_despachos_despachos_sucursales FOREIGN KEY
(id_despacho_sucursal) REFERENCES despachos_sucursales(id_despacho_sucursal),
    CONSTRAINT fk_detalle_despachos_productos FOREIGN KEY (id_producto)
REFERENCES productos(id_producto),

    -- Validaciones de cantidades

```

```

        CONSTRAINT ck_detalle_despachos_envio_positivo CHECK (cantidad_enviada >
0),
        CONSTRAINT ck_detalle_despachos_recibo_positivo CHECK (cantidad_recibida
>= 0.000),

        -- Regla de oro de la física y la logística: No puedes recibir más de lo
que enviaste
        CONSTRAINT ck_recibido_menor_o_igual_enviado CHECK (cantidad_recibida <=
cantidad_enviada)
    );

--TABLA DETALLES_COMPRAS
CREATE TABLE detalles_compras (
    id_detalle_compra BIGSERIAL,
    id_compra BIGINT NOT NULL,
    id_producto BIGINT NOT NULL,
    cantidad_recibida NUMERIC(10,3) NOT NULL,
    costo_unitario_compra NUMERIC(12,2) NOT NULL,

    -- Campos de auditoría (Limpiados caracteres de formato invisibles)
    creado_en TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP NOT NULL,
    modificado_en TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP NOT NULL,
    creado_por VARCHAR(50) DEFAULT 'system' NOT NULL,
    modificado_por VARCHAR(50),

    -- Restricciones (Constraints)
    CONSTRAINT pk_detalle_compras PRIMARY KEY (id_detalle_compra), -- Ajustado
nombre para consistencia
    CONSTRAINT fk_detalle_compras_compras FOREIGN KEY (id_compra) REFERENCES
compras(id_compra),
    CONSTRAINT fk_detalle_compras_productos FOREIGN KEY (id_producto)
REFERENCES productos(id_producto),
    CONSTRAINT ck_compras_cantidad_positiva CHECK (cantidad_recibida > 0),
    CONSTRAINT ck_compras_costo_positiva CHECK (costo_unitario_compra > 0)
);

--TABLA PRODUCTOS_PROMOCIONES
CREATE TABLE productos_promociones (
    id_producto_promocion BIGSERIAL,
    id_producto BIGINT NOT NULL,
    id_promocion BIGINT NOT NULL,

    -- Campos de auditoría y estado
    activo BOOLEAN DEFAULT TRUE NOT NULL,
    creado_en TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP NOT NULL,
    modificado_en TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP NOT NULL,
    creado_por VARCHAR(50) DEFAULT 'system' NOT NULL,
    modificado_por VARCHAR(50),

```

```

-- Restricciones (Constraints)
CONSTRAINT pk_productos_promociones PRIMARY KEY (id_producto_promocion),

-- Llaves foráneas (Conexión con las tablas maestras)
CONSTRAINT fk_prod_promo_productos FOREIGN KEY (id_producto) REFERENCES
productos(id_producto),
CONSTRAINT fk_prod_promo_promociones FOREIGN KEY (id_promocion) REFERENCES
promociones(id_promocion),

-- ¡Regla de oro! Evita asignar la misma promoción al mismo producto más
de una vez
CONSTRAINT uq_producto_promocion UNIQUE (id_producto, id_promocion)
);

--TABLA AUDITORIA_SISTEMA
CREATE TABLE auditoria_sistema(
    id_auditoria_sistema BIGSERIAL,
    id_usuario_sistema BIGINT NOT NULL,
    accion_realizada VARCHAR(50) NOT NULL,
    tabla_afectada VARCHAR (50) NOT NULL,
    registro_id BIGINT NOT NULL,
    valores_anteriores JSONB,
    valores_nuevos JSONB,
    fecha_hora TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP NOT NULL,

--Restriccion
CONSTRAINT pk_auditoria_sistema PRIMARY KEY (id_auditoria_sistema),
CONSTRAINT FK_auditoria_sistema_usuario_sistema FOREIGN KEY
(id_usuario_sistema) REFERENCES usuarios_sistemas(id_usuario_sistema)
);

```

MÓDULO 4

Población y Gestión Inicial de Datos

4.1 Objetivo del módulo

En esta fase del proyecto se realizó la carga inicial de información de la base de datos DataMart 360, incorporando los datos necesarios para simular el funcionamiento operativo de una cadena de supermercados. La población de datos permitió validar el modelo relacional, comprobar las restricciones de integridad y preparar un entorno de pruebas para el desarrollo de consultas, optimización, automatización y análisis de información.

El objetivo principal fue construir un conjunto de datos consistente y representativo que sirviera como base para los módulos posteriores del proyecto.

4.2 Estrategia de carga de datos

La inserción de registros se ejecutó siguiendo un orden lógico que respetara las relaciones entre tablas y las restricciones de claves foráneas. Primero se cargaron las tablas maestras y posteriormente las tablas operacionales y transaccionales.

El orden de carga fue el siguiente:

1. Categorías.
2. Proveedores.
3. Sucursales.
4. Métodos de pago.
5. Empleados.
6. Clientes.
7. Productos.
8. Inventarios.
9. Promociones.
10. Compras.
11. Detalles de compras.
12. Facturas.
13. Detalles de facturas.

Este orden permitió garantizar que todas las claves foráneas apuntaran a registros previamente existentes.

4.3 Población de catálogos maestros

Los catálogos maestros contienen información de referencia utilizada por el resto del sistema. Estos datos presentan baja frecuencia de cambio y son fundamentales para la operación del negocio.

Las principales tablas maestras pobladas fueron:

- categorías
- proveedores
- sucursales
- metodos_pago

Ejemplo de inserción:

```
INSERT INTO categorias (nombre, descripcion)
VALUES ('Lácteos', 'Productos derivados de la leche');
```

La carga de estas tablas permitió establecer la estructura básica sobre la cual operan los procesos de compras, ventas e inventarios.

4.4 Población de datos operacionales

En esta etapa se registró la información necesaria para representar la operación diaria del supermercado.

Las tablas pobladas fueron:

- productos
- empleados
- clientes
- inventarios
- promociones

Ejemplo representativo:

```
INSERT INTO productos (
    nombre,
    id_categoria,
    precio_venta_actual,
    activo
)
VALUES (
    'Leche Entera 1L',
```

```
1,  
1.25,  
TRUE  
);
```

La información cargada permitió representar productos activos, inventario disponible, personal de trabajo y clientes registrados.

4.5 Población de datos transaccionales

Se generaron registros transaccionales para simular el funcionamiento real del negocio, incluyendo procesos de compras y ventas.

Las tablas utilizadas fueron:

- compras
- detalles_compras
- facturas
- detalles_factura

Ejemplo de registro de una factura:

```
INSERT INTO facturas (  
    numero_factura,  
    id_cliente,  
    total_pagar  
)  
VALUES (  
    'F-000001',  
    5,  
    125.50  
);
```

Estos datos fueron fundamentales para realizar posteriormente consultas analíticas, funciones, procedimientos almacenados y triggers.

4.6 Validación de integridad de los datos

Después de la carga inicial se ejecutaron consultas de validación para comprobar la consistencia de la información.

- Verificación de productos sin categoría

```
SELECT *  
FROM productos  
WHERE id_categoria IS NULL;
```

- Verificación de facturas sin cliente

```
SELECT *  
FROM facturas  
WHERE id_cliente IS NULL;
```

- Verificación de inventarios sin producto asociado

```
SELECT *  
FROM inventarios  
WHERE id_producto IS NULL;
```

Estas verificaciones confirmaron que las relaciones entre tablas se encontraban correctamente establecidas.

4.7 Generación de datos de prueba

Con el fin de disponer de un volumen suficiente para pruebas y análisis, se incorporaron registros simulados en las principales tablas del sistema.

Los datos generados incluyeron:

- clientes,
- productos,
- inventarios,
- compras,
- facturas,
- detalles de facturas.

La información fue diseñada para permitir:

- Ejecución de consultas SQL,
- Pruebas de rendimiento,

- Optimización mediante índices,
- Desarrollo de funciones y procedimientos,
- Generación de reportes analíticos.

4.8 Resultados de la carga inicial

La población de datos permitió disponer de una base de datos funcional y preparada para las siguientes etapas del proyecto.

Las principales entidades cargadas fueron:

Tabla	Contenido
categorias	Catálogo de categorías de productos
proveedores	Información de proveedores
sucursales	Estructura de sucursales
empleados	Personal operativo
clientes	Clientes registrados
productos	Catálogo de productos
inventarios	Existencias por sucursal
promociones	Campañas promocionales
compras	Registros de abastecimiento
facturas	Transacciones de venta
detalles_factura	Productos vendidos por factura

4.9 Conclusiones

La carga inicial de datos constituyó una etapa fundamental para el desarrollo de **DataMart 360**, ya que permitió validar el modelo relacional, comprobar las restricciones de integridad referencial y preparar un entorno consistente para el desarrollo de consultas, optimización, automatización y análisis de información.

El conjunto de datos generado sirvió como base para la implementación de índices, vistas, procedimientos almacenados, triggers y reportes analíticos desarrollados en los módulos posteriores del proyecto.

MÓDULO 5

SQL Nivel básico

Estructuras que utilizaremos:

1. Comandos Básicos y Filtrado

1. **SELECT:** Define **qué columnas** quieres mostrar en el resultado. Si usas `SELECT *`, traes todas las columnas disponibles.
2. **WHERE:** **Filtra las filas** según condiciones específicas (ej. solo productos con precio mayor a 25 o usuarios activos).
3. **ORDER BY:** **Ordena los resultados** en función de una o más columnas. Puede ser ascendente (`ASC`, de menor a mayor / A-Z) o descendente (`DESC`, de mayor a menor / Z-A).
4. **LIMIT:** **Restringe el número máximo de filas** devueltas por la consulta (ideal para tops o pagination).
5. **DISTINCT:** **Elimina duplicados** del resultado, mostrando únicamente valores únicos de las columnas elegidas.

2. Agrupamiento y Condicionales

1. **GROUP BY:** **Agrupar filas** que comparten el mismo valor en una o más columnas para poder aplicarles cálculos agregados (ej. contar cuántos empleados hay por cada cargo).
2. **HAVING:** **Filtra los grupos** creados por `GROUP BY`. Funciona como un `WHERE`, pero evalúa los resultados de funciones agregadas (ej. solo mostrar categorías que tengan más de 5 productos).
3. **CASE:** Funciona como un `if/else` **dentro de SQL**. Evalúa condiciones fila por fila y devuelve valores personalizados en una nueva columna (ej. si el precio es > 100 muestra "CARO", de lo contrario "ECONÓMICO").

3. Funciones de Agregación Básicas

Se aplican sobre un conjunto de filas para devolver un solo valor resumen:

1. **COUNT ():** Cuenta el número de filas o de valores no nulos.
2. **SUM ():** Suma los valores numéricos de una columna.
3. **AVG ():** Calcula el promedio de una columna numérica.
4. **MAX ():** Devuelve el valor máximo registrado.
5. **MIN ():** Devuelve el valor mínimo registrado.

4. Uniones de Tablas (JOINS)

Permiten combinar datos de dos o más tablas basándose en una columna relacionada (llave foránea / primaria):

1. **INNER JOIN**: Devuelve **únicamente las filas que coinciden** en ambas tablas. Si un registro de una tabla no tiene correspondencia en la otra, se omite.
2. **LEFT JOIN**: Devuelve **todas las filas de la tabla izquierda**, junto con los datos de la derecha que coincidan. Si no hay coincidencia, llena esos campos con `NULL`.
3. **RIGHT JOIN**: Devuelve **todas las filas de la tabla derecha**, junto con los datos de la izquierda que coincidan. (Es el inverso de `LEFT JOIN`).
4. **FULL JOIN**: Devuelve **todas las filas de ambas tablas**. Coincidan o no, muestra toda la información llenando con `NULL` donde no haya relación.

5. Subconsultas y Operadores Comparativos

1. **Subconsultas**: Consultas integradas dentro de otra consulta principal (generalmente dentro de `WHERE` o `FROM`) para calcular valores dinámicos antes de filtrar la consulta global.
2. **EXISTS**: Comprueba si una subconsulta **devuelve al menos una fila**. Si encuentra coincidencia devuelve `TRUE`, siendo eficiente para validar relaciones.
3. **IN**: Evalúa si un valor coincide con **cualquiera de los elementos dentro de una lista** fija o devuelta por una subconsulta.
4. **ANY**: Compara un valor contra una lista y devuelve verdadero si la condición se cumple **con al menos uno** de los elementos de la subconsulta.
5. **ALL**: Compara un valor contra una lista y devuelve verdadero solo si la condición se cumple **con todos** los elementos de la subconsulta.

6. Operadores de Conjuntos

1. **UNION**: Combina los resultados de dos consultas en una sola lista, **eliminando registros duplicados**.
2. **UNION ALL**: Combina los resultados de dos consultas en una sola lista, **conservando todos los duplicados** (es más rápido que `UNION` al no requerir de duplicación).

7. Estructuras Temporales

1. **CTE (WITH)**: Define una **tabla temporal con nombre** al inicio de la consulta. Sirve para organizar consultas complejas en bloques legibles y reutilizables dentro de la misma sentencia.

8. Funciones de Ventana (Window Functions) y Analíticas

A diferencia de `GROUP BY`, las funciones de ventana realizan cálculos sobre un conjunto de filas relacionadas con la fila actual **sin colapsar las filas individuales**:

2. `ROW_NUMBER()`: Asigna un **número secuencial único** (1, 2, 3...) a cada fila dentro del conjunto de resultados o dentro de cada partición.
3. `RANK()`: Asigna un ranking a cada fila. Si hay valores idénticos, les otorga el mismo puesto y **salta los siguientes puestos** (ej. 1, 2, 2, 4).
4. `DENSE_RANK()`: Asigna un ranking similar a `RANK()`, pero **sin saltar puestos** cuando hay empates (ej. 1, 2, 2, 3).
5. `LAG()`: Accede al **valor de una fila anterior** dentro del mismo grupo de resultados (útil para comparar datos con el registro previo).
6. `LEAD()`: Accede al **valor de una fila posterior** dentro del mismo grupo de resultados (útil para proyectar o comparar con el siguiente registro).
7. `OVER()`: Define la "ventana" sobre la cual actuará una función (permite usar la cláusula `PARTITION BY` para agrupar y `ORDER BY` para ordenar el cálculo contextual).
8. **Consultas Analíticas**: Combinan funciones de ventana y agregaciones avanzadas para resolver métricas de negocio complejas (como comparar una fila con el promedio global o medir porcentajes de participación respecto al total de una sucursal).

SELECT

Ejercicio 1: Muestra todos los datos de la tabla **clientes**.

```
SELECT * FROM clientes;
```

Ejercicio 2: Muestra únicamente el **nombre** y el **precio de venta actual** de todos los productos.

```
SELECT nombre, precio_venta_actual FROM productos;
```

Ejercicio 3: Obtén el **nombre**, la **descripción** y el **stock mínimo de alerta** de los productos.

```
SELECT nombre, descripcion, stock_minimo_alerta FROM productos;
```

Ejercicio 4: Muestra el **nombre completo**, **cargo** y **correo** de todos los empleados.

```
SELECT nombre_completo, cargo, correo FROM empleados;
```

WHERE

Ejercicio 1: Muestra todos los clientes que estén **activos**.

```
SELECT * FROM clientes WHERE activo = 'true';
```

Ejercicio 2: Muestra los productos cuyo precio sea mayor a 25.

```
SELECT * FROM productos WHERE precio_venta_actual > 25;
```

Ejercicio 3: Muestra los empleados cuyo cargo sea '**Cajero**'.

```
SELECT * FROM empleados WHERE cargo = 'Cajero';
```

Ejercicio 4: Muestra las promociones cuyo porcentaje de descuento sea mayor a 20.

```
SELECT * FROM promociones WHERE porcentaje_descuento > 20;
```

ORDER BY

Ejercicio 1: Ordena los productos por **nombre** de forma ascendente.

```
SELECT * FROM productos ORDER BY nombre ASC;
```

Ejercicio 2: Ordena los productos por **precio de venta** de mayor a menor.

```
SELECT * FROM productos ORDER BY precio_venta_actual DESC;
```

Ejercicio 3: Ordena los empleados por **fecha de contratación**, desde el más antiguo hasta el más reciente.

```
SELECT * FROM empleados ORDER BY fecha_contratacion ASC;
```

Ejercicio 4: Ordena los clientes por **nombre completo** de forma descendente.

```
SELECT * FROM clientes ORDER BY nombre_completo DESC;
```

LIMIT

Ejercicio 1: Muestra los primeros 5 **clientes**.

```
SELECT * FROM clientes LIMIT 5;
```

Ejercicio 2: Muestra los primeros 10 **productos**.

```
SELECT * FROM productos LIMIT 10;
```

Ejercicio 3: Obtén las primeras 3 **facturas**.

```
SELECT * FROM facturas LIMIT 3;
```

Ejercicio 4: Muestra los primeros 8 **proveedores**.

```
SELECT * FROM proveedores LIMIT 8;
```

GROUP BY

Ejercicio 1: Cuenta cuántos productos hay por categoría.

```
SELECT id_categoria, COUNT(id_producto) as "Cantidad de productos por categoria" FROM productos GROUP BY id_categoria;
```

Ejercicio 2: Cuenta cuántos empleados existen por cargo.

```
SELECT cargo, COUNT(id_empleado) as "Cantidad de empleados por cargo" FROM empleados GROUP BY cargo;
```

Ejercicio 3: Cuenta cuántas facturas existen por sucursal.

```
SELECT id_sucursal, COUNT(id_factura) as "Cantidad de facturas por sucursal"
FROM facturas GROUP BY id_sucursal;
```

Ejercicio 4: Cuenta cuántos productos hay registrados en cada sucursal dentro de la tabla inventarios.

```
SELECT id_sucursal, COUNT(id_producto) as "Cantidad de productos por
categoria en la tabla inventarios" FROM inventarios GROUP BY id_sucursal;
```

HAVING

Ejercicio 1: Muestra los cargos que tengan **más de 2 empleados**.

```
SELECT cargo, COUNT(*) as "Total"
FROM empleados
GROUP BY cargo
HAVING COUNT(*) > 2;
```

Ejercicio 2: Muestra las categorías que tengan **más de 5 productos**.

```
SELECT id_categoria, COUNT(*) as "Total"
FROM productos
GROUP BY id_categoria
HAVING COUNT(*) > 5;
```

Ejercicio 3: Muestra las sucursales que tengan **más de 20 productos en inventario**.

```
SELECT id_sucursal, COUNT(*) AS "Productos"
FROM inventarios
GROUP BY id_sucursal
HAVING COUNT(*) > 20;
```

Ejercicio 4: Muestra los clientes que tengan **más de una factura registrada**.

```
SELECT id_cliente, COUNT(*) as "Total"
FROM facturas
GROUP BY id_cliente
HAVING COUNT(*) > 1;
```

CASE

Ejercicio 1: Muestra el nombre del producto y crea una columna llamada **Estado**, donde:

1. Si el precio es mayor a 100 → "Caro"
2. En caso contrario → "Económico"

```
SELECT nombre,
CASE
    WHEN precio_venta_actual > 100 THEN 'CARO'
    ELSE 'ECONOMICO'
END AS "Estado"
FROM productos;
```

Ejercicio 2: Muestra el nombre del empleado y clasifica su estado:

1. activo = TRUE → "Activo"
2. activo = FALSE → "Inactivo"

```
SELECT nombre_completo, activo,  
       CASE  
         WHEN activo = TRUE THEN 'Activo'  
         ELSE 'Inactivo'  
       END AS "Estado"  
FROM empleados;
```

Ejercicio 3: Muestra el nombre de la promoción y clasifica:

1. porcentaje_descuento >= 50 → "Descuento Alto"
2. porcentaje_descuento >= 20 → "Descuento Medio"
3. En otro caso → "Descuento Bajo"

```
SELECT nombre_campana,  
       CASE  
         WHEN porcentaje_descuento >=50 THEN 'Descuento Alto'  
         WHEN porcentaje_descuento >=20 THEN 'Descuento Medio'  
         ELSE 'Descuento Bajo'  
       END AS "Descuentos"  
FROM promociones;
```

Ejercicio 4: Muestra el número de factura y crea una columna que indique:

1. total_pagar > 500 → "Compra Grande"
2. En otro caso → "Compra Normal"

```
SELECT numero_factura,  
       CASE  
         WHEN total_pagar > 500 THEN 'Compra grande'  
         ELSE 'Compra normal'  
       END AS "Compra"  
FROM facturas;
```

DISTINCT

Ejercicio 1: Muestra los distintos cargos registrados en la tabla **empleados**.

```
SELECT DISTINCT cargo FROM empleados;
```

Ejercicio 2: Muestra los distintos métodos de pago registrados.

```
SELECT DISTINCT nombre FROM metodos_pago;
```

Ejercicio 3: Muestra los distintos estados de los despachos.

```
SELECT DISTINCT estado FROM despachos_sucursales;
```

Ejercicio 4: Muestra los distintos usuarios que aparecen como **creado_por** en la tabla **productos**.

```
SELECT DISTINCT creado_por FROM productos;
```

🔧 Funciones básicas (*COUNT*, *SUM*, *AVG*, *MAX*, *MIN*)

Ejercicio 1: Cuenta el número total de clientes.

```
SELECT COUNT(*) AS "Total de clientes" FROM clientes;
```

Ejercicio 2: Obtén el precio más alto de los productos.

```
SELECT MAX(precio_venta_actual) AS "Precio mas alto" FROM productos;
```

Ejercicio 3: Calcula el precio promedio de todos los productos.

```
SELECT AVG(precio_venta_actual) AS "Promedio de precio" FROM productos;
```

Ejercicio 4: Calcula la suma total del campo **total_pagar** de todas las facturas.

```
SELECT SUM(total_pagar) AS "Total a pagar facturas" FROM facturas;
```

🔧 Bonus (mezclando conceptos)

Cuando termines los anteriores, intenta resolver estos:

- Muestra los **5 productos más caros**.

```
SELECT nombre, precio_venta_actual AS "Precio_mayor"
FROM productos
ORDER BY precio_venta_actual DESC
LIMIT 5;
```

- Cuenta cuántos productos hay por categoría y muestra únicamente las categorías con más de 10 productos.

```
SELECT id_categoria, COUNT(*) as "Mas de 10 productos"
FROM productos
GROUP BY id_categoria
HAVING COUNT(*) > 10;
```

- Muestra el nombre del producto, su precio y una columna que indique "Premium" si cuesta más de 80 y "Normal" en caso contrario.

```
SELECT nombre, precio_venta_actual,
CASE
    WHEN precio_venta_actual > 80 THEN 'Premium'
    ELSE 'Normal'
END AS "Producto"
FROM productos;
```

- Muestra los distintos cargos de empleados ordenados alfabéticamente.

```
SELECT DISTINCT (cargo)
FROM empleados
ORDER BY cargo ASC;
```

- Obtén el promedio del precio de los productos activos.

```
SELECT AVG(precio_venta_actual) AS "Promedio productos activos"
FROM productos
WHERE activo = 'TRUE';
```

Nivel Intermedio

INNER JOIN

Ejercicio 1: Muestra el nombre del producto, el precio de venta y el nombre de la categoría a la que pertenece.

```
SELECT p.nombre, p.precio_venta_actual, c.nombre
FROM productos p
INNER JOIN categorias c
ON c.id_categoria = p.id_categoria;
```

Ejercicio 2: Muestra el número de factura, la fecha de emisión y el nombre completo del cliente.

```
SELECT f.numero_factura, f.fecha_emision, c.nombre_completo
FROM facturas f
INNER JOIN clientes c
ON c.id_cliente = f.id_cliente
```

Ejercicio 3: Muestra el nombre del producto, la cantidad vendida y el precio unitario histórico de cada detalle de factura.

```
SELECT p.nombre, df.cantidad, df.precio_unitario_historico
FROM productos p
INNER JOIN detalles_facturas df
ON p.id_producto = df.id_producto;
```

Ejercicio 4: Muestra el nombre del usuario del sistema, el rol y el nombre completo del empleado al que pertenece.

```
SELECT us.username, us.rol, e.nombre_completo
FROM usuarios_sistema us
INNER JOIN empleados e
ON e.id_empleado = us.id_empleado;
```

LEFT JOIN

Ejercicio 1: Muestra todos los clientes y, si tienen una tarjeta de fidelización, muestra también el código de su tarjeta.

```
SELECT c.*, tf.codigo_tarjeta
FROM clientes c
LEFT JOIN tarjetas_fidelizacion tf
ON c.id_cliente = tf.id_cliente;
```

Ejercicio 2: Muestra todas las categorías junto con los productos que pertenecen a cada una, incluyendo las categorías que no tengan productos.

```
SELECT c.nombre AS "Categoría", p.nombre AS "Producto"
FROM categorias c
LEFT JOIN productos p
ON c.id_categoria = p.id_categoria;
```

Ejercicio 3: Muestra todos los productos y, si tienen promociones asignadas, muestra el nombre de la campaña.

```
SELECT p.nombre, pr.nombre_campana
FROM productos p
LEFT JOIN productos_promociones pp
ON p.id_producto = pp.id_producto
LEFT JOIN promociones pr
ON pr.id_promocion = pp.id_promocion;
```

Ejercicio 4: Muestra todas las sucursales y los inventarios registrados en cada una, incluso si alguna sucursal no tiene inventario.

```
SELECT s.nombre, i.id_inventario
FROM sucursales s
LEFT JOIN inventarios i
ON s.id_sucursal = i.id_sucursal;
```

RIGHT JOIN

Ejercicio 1: Muestra todos los clientes y sus facturas, incluyendo aquellos clientes que todavía no tengan ninguna factura.

```
SELECT c.nombre_completo, f.numero_factura
FROM facturas f
RIGHT JOIN clientes c
ON f.id_cliente = c.id_cliente;
```

Ejercicio 2: Muestra todas las categorías y los productos relacionados, usando RIGHT JOIN.

```
SELECT c.nombre AS "Categoría", p.nombre AS "Producto"
FROM productos p
RIGHT JOIN categorias c
ON p.id_categoria = c.id_categoria;
```

Ejercicio 3: Muestra todos los empleados y sus usuarios del sistema, incluyendo empleados que aún no tengan usuario.

```
SELECT e.nombre_completo, u.username
FROM usuarios_sistema u
RIGHT JOIN empleados e
ON u.id_empleado = e.id_empleado;
```

Ejercicio 4: Muestra todos los productos y sus promociones utilizando RIGHT JOIN.

```
SELECT p.nombre, pr.nombre_campana
FROM productos p
RIGHT JOIN productos_promociones pp
ON p.id_producto = pp.id_producto
RIGHT JOIN promociones pr
ON pr.id_promocion = pp.id_promocion;
```

FULL JOIN

Ejercicio 1: Muestra todos los clientes y todas las tarjetas de fidelización, incluso si alguno no tiene relación con el otro.

```
SELECT c.nombre_completo, t.codigo_tarjeta
FROM tarjetas_fidelizacion t
FULL JOIN clientes c
ON t.id_cliente = c.id_cliente;
```

Ejercicio 2: Muestra todas las categorías y todos los productos, existan o no relaciones entre ellos.

```
SELECT c.nombre, p.nombre
FROM categorias c
FULL JOIN productos p
ON c.id_categoria = p.id_categoria;
```

Ejercicio 3: Muestra todos los empleados y todos los usuarios del sistema.

```
SELECT e.nombre_completo, u.username
FROM empleados e
FULL JOIN usuarios_sistema u
ON e.id_empleado = u.id_empleado;
```

Ejercicio 4: Muestra todos los productos y todas las promociones.

```
SELECT p.nombre, pr.nombre_campana
FROM productos p
FULL JOIN productos_promociones pp
ON p.id_producto = pp.id_producto
FULL JOIN promociones pr
ON pr.id_promocion = pp.id_promocion;
```

Subconsultas

Ejercicio 1: Muestra los productos cuyo precio sea mayor que el precio promedio de todos los productos.

```
SELECT nombre, precio_venta_actual
FROM productos
WHERE precio_venta_actual > (
    SELECT AVG(precio_venta_actual)
    FROM productos
);
```

Ejercicio 2: Muestra los clientes que hayan realizado la factura con el monto más alto.

```
SELECT c.nombre_completo, f.numero_factura, f.total_pagar
FROM facturas f
INNER JOIN clientes c ON f.id_cliente = c.id_cliente
WHERE f.total_pagar = (
    SELECT MAX(total_pagar)
    FROM facturas
);
```

Ejercicio 3: Muestra los empleados cuya fecha de contratación sea la más antigua.

```
SELECT nombre_completo, fecha_contratacion
FROM empleados
WHERE fecha_contratacion = (
    SELECT MIN(fecha_contratacion)
    FROM empleados
);
```

Ejercicio 4: Muestra los productos cuyo precio sea igual al precio más alto registrado.

```
SELECT nombre, precio_venta_actual
FROM productos
WHERE precio_venta_actual = (
    SELECT MAX(precio_venta_actual)
    FROM productos
);
```

EXISTS

Ejercicio 1: Muestra los clientes que tengan al menos una factura registrada.

```
SELECT c.nombre_completo, c.correo_electronico
FROM clientes c
WHERE EXISTS (
    SELECT 1
    FROM facturas f
    WHERE f.id_cliente = c.id_cliente
);
```

Ejercicio 2: Muestra las categorías que tengan al menos un producto.

```
SELECT c.nombre
FROM categorias c
WHERE EXISTS (
    SELECT 1
    FROM productos p
    WHERE p.id_categoria = c.id_categoria
);
```

Ejercicio 3: Muestra las sucursales que tengan inventario registrado.

```
SELECT s.nombre
FROM sucursales s
WHERE EXISTS (
    SELECT 1
    FROM inventarios i
    WHERE i.id_sucursal = s.id_sucursal
);
```

Ejercicio 4: Muestra los productos que hayan sido vendidos al menos una vez.

```
SELECT p.nombre
FROM productos p
WHERE EXISTS (
    SELECT 1
    FROM detalles_facturas d
    WHERE d.id_producto = p.id_producto
);
```

IN

Ejercicio 1: Muestra los productos cuya categoría sea 1, 3 o 5.

```
SELECT *
FROM productos
WHERE id_categoria IN (1,3,5);
```

Ejercicio 2: Muestra las facturas realizadas en las sucursales 2, 4 y 6.

```
SELECT *
FROM facturas
WHERE id_sucursal IN (2,4,6);
```

Ejercicio 3: Muestra los empleados cuyo cargo sea "Cajero", "Administrador" o "Supervisor".

```
SELECT *
FROM empleados
WHERE cargo IN ('Cajero', 'Administrador', 'Supervisor');
```

Ejercicio 4: Muestra las promociones cuyo porcentaje de descuento sea 10, 20 o 50.

```
SELECT *
FROM promociones
WHERE porcentaje_descuento IN (10,20,50);
```

ANY

Ejercicio 1: Muestra los productos cuyo precio sea mayor que el precio de al menos un producto de la categoría 1.

```
SELECT nombre, precio_venta_actual
FROM productos
WHERE precio_venta_actual > ANY (
    SELECT precio_venta_actual
    FROM productos
    WHERE id_categoria = 1
);
```

Ejercicio 2: Muestra las facturas cuyo total a pagar sea mayor que alguna factura emitida en la sucursal 2.

```
SELECT numero_factura, id_sucursal, total_pagar
FROM facturas
WHERE total_pagar > ANY (
    SELECT total_pagar
    FROM facturas
    WHERE id_sucursal = 2
);
```

Ejercicio 3: Muestra los productos cuyo precio sea menor que algún producto de la categoría 3.

```
SELECT nombre, precio_venta_actual, id_categoria
FROM productos
WHERE precio_venta_actual < ANY (
    SELECT precio_venta_actual
    FROM productos
    WHERE id_categoria = 3
);
```

Ejercicio 4: Muestra las promociones cuyo porcentaje sea mayor que algún descuento existente.

```
SELECT nombre_campana, porcentaje_descuento
FROM promociones
WHERE porcentaje_descuento > ANY (
    SELECT monto_descuento_fijo
    FROM promociones
);
```

ALL

Ejercicio 1: Muestra los productos cuyo precio sea mayor que todos los productos de la categoría 2.

```
SELECT nombre, precio_venta_actual, id_categoria
FROM productos
WHERE precio_venta_actual > ALL (
    SELECT precio_venta_actual
    FROM productos
    WHERE id_categoria = 2
);
```

Ejercicio 2: Muestra las facturas cuyo total sea mayor que todas las facturas emitidas en la sucursal 1.

```
SELECT numero_factura, total_pagar, id_sucursal
FROM facturas
WHERE total_pagar > ALL (
    SELECT total_pagar
    FROM facturas
    WHERE id_sucursal = 1
);
```

Ejercicio 3: Muestra los productos cuyo precio sea menor que todos los productos de la categoría 5.

```
SELECT nombre, precio_venta_actual, id_categoria
FROM productos
WHERE precio_venta_actual < ALL (
    SELECT precio_venta_actual
    FROM productos
    WHERE id_categoria = 5
);
```

Ejercicio 4: Muestra las promociones cuyo porcentaje de descuento sea mayor que todos los descuentos menores a 30%.

```
SELECT nombre_campana, porcentaje_descuento
FROM promociones
WHERE porcentaje_descuento > ALL (
    SELECT porcentaje_descuento
    FROM promociones
    WHERE porcentaje_descuento < 30
);
```

UNION

Ejercicio 1: Muestra una lista con los nombres de todos los clientes y todos los empleados, eliminando duplicados.

```
SELECT nombre_completo
FROM clientes
UNION
SELECT nombre_completo
FROM empleados;
```

Ejercicio 2: Muestra todos los correos electrónicos de clientes y proveedores sin repetir valores.

```
SELECT correo_electronico
FROM clientes
UNION
SELECT correo
FROM proveedores;
```

Ejercicio 3: Muestra todas las direcciones registradas en clientes y sucursales.

```
SELECT direccion
FROM clientes
UNION
SELECT direccion
FROM sucursales;
```

Ejercicio 4: Muestra todos los teléfonos registrados en clientes y empleados.

```
SELECT telefono
FROM clientes
UNION
SELECT telefono
FROM empleados;
```

11. UNION ALL

Ejercicio 1: Muestra todos los nombres de clientes y empleados, permitiendo registros repetidos.

```
SELECT nombre_completo
FROM clientes
UNION ALL
SELECT nombre_completo
FROM empleados;
```

Ejercicio 2: Muestra todos los teléfonos de clientes y proveedores, incluyendo duplicados.

```
SELECT telefono
FROM clientes
UNION ALL
SELECT telefono
FROM proveedores;
```

Ejercicio 3: Muestra todos los correos de empleados y proveedores.

```
SELECT correo
FROM empleados
UNION ALL
SELECT correo
FROM proveedores;
```

Ejercicio 4: Muestra todas las direcciones de clientes y proveedores.

```
SELECT direccion
FROM clientes
UNION ALL
SELECT direccion
FROM proveedores;
```

12. CTE (WITH)

Ejercicio 1: Crea una CTE llamada productos_caros que contenga los productos con precio mayor a 25 y luego muestra todos sus datos.

```
WITH productos_caros AS (
    SELECT nombre, precio_venta_actual
    FROM productos
    WHERE precio_venta_actual > 25
)
SELECT *
FROM productos_caros;
```

Ejercicio 2: Crea una CTE que calcule el total de facturas por cliente y muestra únicamente aquellos clientes con más de 5 facturas.

```
WITH total_facturas AS (
    SELECT id_cliente, COUNT (id_factura) AS "total"
    FROM facturas
    GROUP BY id_cliente
)
SELECT *
FROM total_facturas
WHERE total > 5;
```

Ejercicio 3: Crea una CTE que calcule el promedio del precio de los productos y luego muestra los productos cuyo precio sea superior a ese promedio.

```
WITH promedio_general AS (
    SELECT AVG (precio_venta_actual) AS promedio
    FROM productos
)
SELECT p.nombre, p.precio_venta_actual
FROM productos p, promedio_general pg
WHERE p.precio_venta_actual > pg.promedio;
```

Ejercicio 4: Crea una CTE que cuente cuántos productos existen por categoría y muestra únicamente las categorías con más de 10 productos.

```
WITH productos_categoria AS (  
    SELECT id_categoria, COUNT (*) AS "total_productos"  
    FROM productos  
    GROUP BY id_categoria  
)  
SELECT id_categoria, total_productos  
FROM productos_categoria  
WHERE total_productos > 10;
```

Nivel Avanzado

ROW_NUMBER

Ejercicio 1: Numera todos los productos ordenados por precio de venta, del más caro al más barato.

```
SELECT nombre, precio_venta_actual,  
ROW_NUMBER() OVER (  
    ORDER BY precio_venta_actual DESC  
) AS posicion  
FROM productos;
```

Ejercicio 2: Numera las facturas de cada cliente según la fecha de emisión, comenzando desde 1 para cada cliente.

```
SELECT id_cliente, numero_factura, fecha_emision,  
ROW_NUMBER() OVER (  
    PARTITION BY id_cliente  
    ORDER BY fecha_emision ASC  
) AS numeracion  
FROM facturas;
```

Ejercicio 3: Numera los productos dentro de cada categoría ordenándolos por nombre.

```
SELECT id_producto, nombre, id_categoria,  
ROW_NUMBER() OVER (  
    PARTITION BY id_categoria  
    ORDER BY nombre ASC  
) AS numeracion  
FROM productos;
```

Ejercicio 4: Numera los empleados según su fecha de contratación, del más antiguo al más reciente.

```
SELECT nombre_completo, fecha_contratacion,  
       ROW_NUMBER() OVER(  
         ORDER BY fecha_contratacion ASC  
       ) AS numeracion_empleados  
FROM empleados;
```

Ejercicio 5: Numera las compras realizadas por cada proveedor ordenadas por fecha de compra.

```
SELECT id_compra, fecha_compra, id_proveedor,  
       ROW_NUMBER() OVER(  
         PARTITION BY id_proveedor  
         ORDER BY fecha_compra ASC  
       ) AS compras_realizadas  
FROM compras;
```

RANK

Ejercicio 1: Asigna un ranking a los productos según su precio, del mayor al menor.

```
SELECT  
  id_producto, nombre, precio_venta_actual,  
  RANK() OVER (  
    ORDER BY precio_venta_actual DESC  
  ) AS ranking  
FROM productos;
```

Ejercicio 2: Obtén el ranking de las facturas según el total a pagar.

```
SELECT  
  numero_factura, total_pagar,  
  RANK() OVER (  
    ORDER BY total_pagar DESC  
  ) AS ranking_facturas  
FROM facturas;
```

Ejercicio 3: Asigna un ranking a los clientes según la cantidad de facturas que poseen.

```
SELECT  
  id_cliente,  
  COUNT(*) AS cantidad_facturas,  
  RANK() OVER (  
    ORDER BY COUNT(*) DESC  
  ) AS ranking_clientes  
FROM facturas  
GROUP BY id_cliente;
```

Ejercicio 4: Obtén el ranking de las promociones según el porcentaje de descuento.

```
SELECT
    id_promocion, nombre_campana, porcentaje_descuento,
    RANK() OVER (
        ORDER BY porcentaje_descuento DESC
    ) AS ranking_promociones
FROM promociones;
```

Ejercicio 5: Asigna un ranking a las sucursales según el total de productos registrados en inventario.

```
SELECT
    id_sucursal,
    SUM(stock_actual) AS total_inventario,
    RANK() OVER (
        ORDER BY SUM(stock_actual) DESC
    ) AS ranking_sucursales
FROM inventarios
GROUP BY id_sucursal;
```

DENSE_RANK

Ejercicio 1: Asigna un ranking sin saltos a los productos según el precio.

```
SELECT id_producto, nombre, precio_venta_actual,
    DENSE_RANK() OVER(
        ORDER BY precio_venta_actual DESC
    ) AS sin_saltos
FROM productos;
```

Ejercicio 2: Clasifica las promociones según su descuento utilizando DENSE_RANK.

```
SELECT id_promocion, nombre_campana, porcentaje_descuento,
    DENSE_RANK() OVER (
        ORDER BY porcentaje_descuento DESC
    ) AS descuento
FROM promociones;
```

Ejercicio 3: Obtén un ranking de clientes según el número de facturas.

```
SELECT id_cliente, count(*),
    DENSE_RANK() OVER(
        ORDER BY count(*) DESC
    ) AS factura
FROM facturas
GROUP BY id_cliente;
```

Ejercicio 4: Clasifica las categorías según la cantidad de productos.

```
SELECT id_categoria, COUNT(*),
    DENSE_RANK() OVER(
        ORDER BY COUNT(*) DESC
```

```

        )AS clasificacion
FROM productos
GROUP BY id_categoria;

```

Ejercicio 5: Obtén un ranking de proveedores según el número de compras realizadas.

```

SELECT id_proveedor, COUNT(*),
       DENSE_RANK() OVER(
         ORDER BY COUNT(*) DESC
       )AS proveedoresRANK
FROM compras
GROUP BY id_proveedor;

```

LAG

Ejercicio 1: Para cada factura, muestra el total a pagar y el total de la factura inmediatamente anterior.

```

SELECT id_factura, numero_factura, total_pagar,
       LAG(total_pagar, 1, 0) OVER (
         ORDER BY id_factura ASC
       ) AS facturaAnterior
FROM facturas;

```

Ejercicio 2: Para cada compra de un proveedor, muestra la fecha de compra anterior.

```

SELECT id_compra, id_proveedor, fecha_compra,
       LAG(fecha_compra, 1) OVER (
         PARTITION BY id_proveedor
         ORDER BY fecha_compra ASC
       ) AS fechaAnterior
FROM compras;

```

Ejercicio 3: Para cada producto ordenado por precio, muestra el precio del producto anterior.

```

SELECT
  id_producto, nombre, precio_venta_actual,
  LAG(precio_venta_actual, 1, 0) OVER (
    ORDER BY precio_venta_actual ASC
  ) AS precio_producto_anterior
FROM productos;

```

Ejercicio 4: Para cada empleado ordenado por fecha de contratación, muestra la fecha del empleado contratado anteriormente.

```

SELECT
  id_empleado, nombre_completo, fecha_contratacion,
  LAG(fecha_contratacion, 1) OVER (
    ORDER BY fecha_contratacion ASC
  ) AS fecha_contratacion_anterior

```

```
FROM empleados;
```

Ejercicio 5: Para cada promoción ordenada por fecha de inicio, muestra la promoción anterior.

```
SELECT
    id_promocion, nombre_campana, fecha_inicio,
    LAG(nombre_campana, 1) OVER (
        ORDER BY fecha_inicio ASC
    ) AS promocion_anterior
FROM promociones;
```

LEAD

Ejercicio 1: Muestra el total de cada factura y el total de la siguiente factura.

```
SELECT
    id_factura, total_pagar,
    LEAD(total_pagar, 1, 0) OVER(
        ORDER BY id_factura ASC
    ) AS Total
FROM facturas;
```

Ejercicio 2: Muestra cada compra junto con la fecha de la siguiente compra del mismo proveedor.

```
SELECT
    id_compra, id_proveedor, fecha_compra,
    LEAD(fecha_compra, 1) OVER (
        PARTITION BY id_proveedor
        ORDER BY fecha_compra ASC
    ) AS siguiente_fecha_compra
FROM compras;
```

Ejercicio 3: Para cada producto, muestra el precio del siguiente producto ordenado por precio.

```
SELECT
    id_producto, nombre, precio_venta_actual,
    LEAD(precio_venta_actual, 1, 0) OVER (
        ORDER BY precio_venta_actual ASC
    ) AS siguiente_precio
FROM productos;
```

Ejercicio 4: Muestra cada promoción junto con la fecha de inicio de la siguiente promoción.

```
SELECT
    id_promocion, nombre_campana, fecha_inicio,
    LEAD(fecha_inicio, 1) OVER (
        ORDER BY fecha_inicio ASC
    ) AS siguiente_fecha_inicio
FROM promociones;
```

Ejercicio 5: Para cada empleado, muestra la fecha de contratación del siguiente empleado.

```
SELECT
    id_empleado, nombre_completo, fecha_contratacion,
    LEAD(fecha_contratacion, 1) OVER (
        ORDER BY fecha_contratacion ASC
    ) AS siguiente_fecha_contratacion
FROM empleados;
```

OVER

Ejercicio 1: Muestra cada producto junto con el precio promedio de todos los productos.

```
SELECT
    id_producto, nombre, precio_venta_actual,
    AVG(precio_venta_actual) OVER () AS precio_promedio_global
FROM productos;
```

Ejercicio 2: Muestra cada factura junto con el total acumulado de ventas.

```
SELECT
    id_factura, fecha_emision, total_pagar,
    SUM(total_pagar) OVER (
        ORDER BY fecha_emision ASC
    ) AS total_ventas_acumulado
FROM facturas;
```

Ejercicio 3: Muestra cada producto junto con el precio máximo existente en toda la tabla.

```
SELECT
    id_producto, nombre, precio_venta_actual,
    MAX(precio_venta_actual) OVER () AS precio_maximo_global
FROM productos;
```

Ejercicio 4: Muestra cada compra junto con el promedio del total pagado.

```
SELECT
    id_compra, total_pagado,
    AVG(total_pagado) OVER () AS promedio_compras_global
FROM compras;
```

Ejercicio 5: Muestra cada empleado junto con la cantidad total de empleados.

```
SELECT
    id_empleado, nombre_completo,
    COUNT(*) OVER () AS total_empleados_empresa
FROM empleados;
```

CONSULTAS ANALÍTICAS

Ejercicio 1: Para cada factura muestra:

- Número de factura
- Cliente
- Total, a pagar
- Promedio de compra del cliente
- Diferencia entre esa factura y el promedio del cliente.

```
SELECT
    numero_factura, id_cliente, total_pagar,
    -- 1. Promedio de compras de ESTE cliente:
    AVG(total_pagar) OVER (
        PARTITION BY id_cliente
    ) AS promedio_cliente,
    -- 2. Diferencia (Factura Actual - Promedio):
    total_pagar - AVG(total_pagar) OVER (
        PARTITION BY id_cliente
    ) AS diferencia_vs_promedio
FROM facturas;
```

Ejercicio 2: Para cada producto muestra:

- Nombre
- Precio
- Ranking dentro de su categoría
- Precio promedio de la categoría.

```
SELECT
    nombre,
    precio_venta_actual AS precio,
    -- 1. Ranking de precio dentro de su categoría:
    DENSE_RANK() OVER (
        PARTITION BY id_categoria
        ORDER BY precio_venta_actual DESC
    ) AS ranking_categoria,
    -- 2. Precio promedio de la categoría:
    AVG(precio_venta_actual) OVER (
        PARTITION BY id_categoria
    ) AS promedio_categoria
FROM productos;
```

Ejercicio 3: Para cada sucursal muestra:

- Producto
- Stock
- Stock total de la sucursal
- Porcentaje que representa ese producto respecto al inventario total de la sucursal.

```
SELECT
    id_producto, id_sucursal,
    stock_actual AS stock,
    -- 1. Stock total acumulado en la sucursal:
    SUM(stock_actual) OVER (
        PARTITION BY id_sucursal
    ) AS stock_total_sucursal,
    -- 2. Porcentaje que representa respecto a la sucursal:
    ROUND(
        (stock_actual::numeric / SUM(stock_actual)
        OVER (PARTITION BY id_sucursal)) * 100, 2
    ) AS porcentaje_inventario
FROM inventarios;
```

(Nota: Usamos `::numeric` para asegurar que PostgreSQL haga la división con decimales de forma exacta).

Ejercicio 4: Para cada proveedor muestra:

- Compra
- Total, pagado
- Compra anterior
- Diferencia respecto a la compra anterior.

```
SELECT
    id_compra, id_proveedor, total_pagado,
    -- 1. Monto pagado en la compra anterior de este mismo proveedor:
    LAG(total_pagado, 1) OVER (
        PARTITION BY id_proveedor
        ORDER BY fecha_compra ASC
    ) AS compra_anterior,
    -- 2. Diferencia respecto a la compra anterior:
    total_pagado - LAG(total_pagado, 1) OVER (
        PARTITION BY id_proveedor
        ORDER BY fecha_compra ASC
    ) AS diferencia_compra_anterior
FROM compras;
```

Ejercicio 5: Para cada cliente muestra:

- Número de facturas
- Monto total comprado
- Ranking de clientes por monto comprado
- Porcentaje del total de ventas que representa ese cliente.

```
SELECT
    id_cliente,
-- 1. Cantidad de facturas del cliente:
    COUNT(id_factura) AS numero_facturas,
-- 2. Monto total comprado por el cliente:
    SUM(total_pagar) AS total_comprado,
-- 3. Ranking de clientes por monto total comprado:
    DENSE_RANK() OVER (
        ORDER BY SUM(total_pagar) DESC
    ) AS ranking_cliente,
-- 4. Porcentaje que representa sobre la venta total global de la empresa:
    ROUND(
        (SUM(total_pagar) / SUM(SUM(total_pagar)) OVER ())) * 100, 2
    ) AS porcentaje_ventas_global
FROM facturas
GROUP BY id_cliente;
```

MÓDULO 6

Administración y Optimización de PostgreSQL

Introducción

En este módulo se implementaron técnicas de administración, optimización, automatización y seguridad sobre la base de datos PostgreSQL de DataMart 360. El objetivo fue mejorar el rendimiento de las consultas, garantizar la integridad de la información, automatizar procesos de negocio y proteger el acceso a los datos mediante roles y permisos.

6.1 Índices

¿Qué son?

Los índices son estructuras que permiten localizar información de manera más rápida dentro de una tabla, evitando recorridos completos de los registros.

Aplicación en DataMart 360

Se implementaron índices sobre columnas frecuentemente utilizadas en búsquedas y filtros, tales como:

- número de identificación de clientes
- fecha de compra
- código de barras de productos
- fecha de emisión de facturas
- nombre de campañas promocionales

Ejercicios desarrollados

Índices simples

```
CREATE INDEX idx_clientes_nombre  
ON clientes(nombre_completo);  
CREATE INDEX idx_productos_codigo  
ON productos(codigo_barras);
```

Índices compuestos

```
CREATE INDEX idx_facturas_cliente_fecha  
ON facturas(id_cliente, fecha_emision);
```

Beneficios

- Reducción del tiempo de búsqueda.

- Mejor rendimiento en reportes.
- Optimización de filtros y ordenamientos.

6.2 Análisis de Rendimiento

¿Qué es?

Proceso mediante el cual se estudia la forma en que PostgreSQL ejecuta una consulta para identificar posibles optimizaciones.

Herramientas utilizadas

- EXPLAIN: Muestra el plan de ejecución estimado.
- EXPLAIN ANALYZE: Ejecuta la consulta y muestra estadísticas reales.

Ejercicios desarrollados

```
EXPLAIN
SELECT *
FROM productos
WHERE codigo_barras = '123456789';
```

```
EXPLAIN ANALYZE
SELECT *
FROM facturas
WHERE fecha_emision >= '2026-01-01';
```

Conceptos analizados

- Seq Scan
- Index Scan
- Cost
- Actual Time
- Execution Time

6.3 Optimización de Consultas

¿Qué es?

Conjunto de buenas prácticas destinadas a reducir el tiempo de respuesta y el consumo de recursos.

Técnicas aplicadas

- Evitar SELECT *

Incorrecto:

```
SELECT *
FROM productos;
```

Correcto:

```
SELECT nombre, precio_venta_actual  
FROM productos;
```

- Filtrar con WHERE

```
SELECT *  
FROM facturas  
WHERE id_cliente = 25;
```

- Utilizar LIMIT

```
SELECT nombre  
FROM productos  
ORDER BY precio_venta_actual DESC  
LIMIT 10;
```

- Evitar funciones sobre columnas indexadas

Incorrecto:

```
WHERE EXTRACT(MONTH FROM fecha_emision) = 6;
```

Correcto:

```
WHERE fecha_emision >= '2026-06-01'  
AND fecha_emision < '2026-07-01';
```

6.4 Vistas (Views)

¿Qué son?

Consultas almacenadas que se comportan como tablas virtuales.

Aplicación en DataMart 360

Se utilizaron para simplificar consultas y facilitar la creación de reportes.

Ejercicios desarrollados

- Clientes activos

```
CREATE VIEW vista_clientes_activos AS  
SELECT nombre_completo,  
       correo_electronico,  
       telefono  
FROM clientes
```

```
WHERE activo = TRUE;
```

- Productos por categoría

```
CREATE VIEW vista_productos_categoria AS
SELECT p.nombre AS producto,
       c.nombre AS categoria
FROM productos p
INNER JOIN categorias c
ON p.id_categoria = c.id_categoria;
```

6.5 Vistas Materializadas

¿Qué son?

Vistas que almacenan físicamente los resultados de una consulta.

Aplicación en DataMart 360

Optimización de reportes y dashboards analíticos.

Ejercicio desarrollado

```
CREATE MATERIALIZED VIEW mv_ventas_mensuales AS
SELECT
    fecha_emision,
    SUM(total_pagar)
FROM facturas
GROUP BY fecha_emision;
```

Actualización:

```
REFRESH MATERIALIZED VIEW mv_ventas_mensuales;
```

6.6 Gestión de Transacciones

¿Qué son?

Conjunto de operaciones ejecutadas como una única unidad de trabajo.

Objetivo

Garantizar la integridad de la información.

Conceptos estudiados

- **BEGIN:** Inicia una transacción.
- **COMMIT:** Confirma los cambios.

- **ROLLBACK:** Revierte todos los cambios.
- **SAVEPOINT:** Permite regresar a un punto específico de la transacción.

Ejercicio desarrollado

```
BEGIN;
INSERT INTO facturas (...);
SAVEPOINT factura_creada;
INSERT INTO detalles_factura (...);
ROLLBACK TO SAVEPOINT factura_creada;
COMMIT;
```

6.7 Seguridad y Control de Acceso

¿Qué es?

Administración de usuarios, roles y permisos dentro de PostgreSQL.

Conceptos estudiados

- **Usuarios:** Cuentas con acceso a la base de datos.
- **Roles:** Conjuntos de permisos reutilizables.
- **GRANT:** Asigna permisos.
- **REVOKE:** Revoca permisos.

Ejercicios desarrollados

Crea un rol

```
CREATE ROLE supervisor;
```

Crea un usuario

```
CREATE USER ana
WITH PASSWORD 'Ana2026';
```

Asigna permisos al rol

```
GRANT SELECT
ON facturas
TO analista;
```

Asigna un rol a un usuario

```
GRANT analista TO ana;
```

6.8 Estrategia de Respaldo y Recuperación

¿Qué es?

Conjunto de procedimientos destinados a proteger la información ante fallos o pérdidas de datos.

Herramientas utilizadas

- **pg_dump**: Generación de respaldos.
- **pg_restore**: Restauración de respaldos.

Ejercicios desarrollados

- Respaldo de la BD completa

```
pg_dump -U postgres -d datamart360 -f backup_julio.sql
```

- Respaldo de tabla

```
pg_dump -U postgres -d datamart360 -t clientes -f clientes.sql
```

- Respaldo de esquema

```
pg_dump -U postgres -d datamart360 --schema-only -f esquema.sql
```

- Respaldo de datos

```
pg_dump -U postgres -d datamart360 --data-only -f datos.sql
```

- Restaurar un respaldo

```
psql -U postgres -d datamart360 -f backup.sql
```

- **FORMATO PERSONALIZADOS**, custom format (Es mas profesional, es mas rápido, comprime los datos y permite hacer restauraciones parciales)

```
pg_dump -U postgres -d datamart360 -F c -f backup.backup
```

- Restaurarlo

```
pg_restore -U postgres -d datamart360 backup.backup
```

6.9 Gestión de Secuencias

¿Qué son?

Objetos de PostgreSQL que generan números consecutivos automáticamente.

Aplicación en DataMart 360

- números de factura
- números de compra
- números de despacho

Ejercicios desarrollados

```
CREATE SEQUENCE sec_compras
START 500
INCREMENT 1;
SELECT nextval('sec_compras');
SELECT currval('sec_compras');
```

Reiniciar la secuencia

```
ALTER SEQUENCE sec_compras RESTART WITH 1;
```

Eliminar la secuencia

```
DROP SEQUENCE sec_compras;
```

6.10 Funciones y Procedimientos Almacenados

Funciones

¿Qué son?

Bloques de código reutilizables que reciben parámetros y devuelven un resultado.

Ejercicios desarrollados

```
CREATE FUNCTION sumar(
    p_num1 NUMERIC,
    p_num2 NUMERIC
)
RETURNS NUMERIC
CREATE FUNCTION calcular_descuento(...)
CREATE FUNCTION obtener_stock_producto(...)
```

Procedimientos

¿Qué son?

Procesos almacenados que ejecutan múltiples operaciones de negocio.

Ejercicios desarrollados

```
CREATE PROCEDURE actualizar_stock(...)
CREATE PROCEDURE desactivar_cliente(...)
CREATE PROCEDURE registrar_venta(...)
```

6.11 Triggers

¿Qué son?

Mecanismos automáticos que ejecutan acciones cuando ocurre un evento sobre una tabla.

Aplicación en DataMart 360

- Validación de stock
- Actualización automática de inventario
- Auditoría de movimientos
- Control de reposición

Ejercicios desarrollados

Descontar inventario

```
CREATE TRIGGER trg_descontar_inventario  
AFTER INSERT  
ON detalles_factura  
FOR EACH ROW;
```

Registrar movimientos

```
CREATE TRIGGER trg_registrar_movimiento  
AFTER INSERT  
ON detalles_factura  
FOR EACH ROW;
```

Aumentar stock

```
CREATE TRIGGER trg_aumentar_stock  
AFTER INSERT  
ON detalles_compras  
FOR EACH ROW;
```

Conclusiones del Módulo

Durante este módulo se implementaron mecanismos avanzados de administración, seguridad, automatización y optimización en PostgreSQL. Se desarrollaron índices, vistas, transacciones, funciones, procedimientos almacenados y triggers que permitieron mejorar el rendimiento del sistema y automatizar procesos críticos de DataMart 360. Además, se estableció una estrategia de seguridad basada en roles y una política de respaldo y recuperación para garantizar la disponibilidad e integridad de la información.

MÓDULO 7

Excel para Análisis de datos

Introducción

En este módulo Excel se utilizó como una herramienta de análisis de datos orientada a un entorno empresarial, trabajando con información exportada desde PostgreSQL. El objetivo fue transformar, limpiar, analizar y visualizar los datos de DataMart 360, aplicando técnicas utilizadas por analistas de datos para la generación de reportes, indicadores y dashboards ejecutivos.

A diferencia de un curso tradicional de Excel, el enfoque de este módulo estuvo orientado al análisis de información, la automatización de procesos y la preparación de datos para la toma de decisiones.

7.1 Importación y conexión de datos

¿Qué es?

Proceso mediante el cual los datos generados en PostgreSQL son exportados e incorporados a Excel para su análisis.

Aplicación en DataMart 360

Se importaron archivos CSV provenientes de las principales tablas del sistema, incluyendo:



Actividades desarrolladas

- Exportación de datos desde PostgreSQL.
- Importación de archivos CSV.
- Conversión de rangos a tablas de Excel.
- Actualización y administración de conexiones.
- Transformación de datos.

7.2 Limpieza y preparación de datos

¿Qué es?

Conjunto de técnicas destinadas a mejorar la calidad de los datos antes de realizar análisis.

Aplicación en DataMart 360

Se prepararon los datos de clientes, productos, ventas y demás tablas para garantizar consistencia y confiabilidad en los reportes.

Temas trabajados

- Eliminación de duplicados.
- Manejo de valores nulos.
- Separación y combinación de columnas.
- Texto en columnas.
- Eliminación de espacios y caracteres especiales.
- Conversión de formatos.
- Normalización de fechas.

7.3 Power Query

¿Qué es?

Herramienta de Excel para importar, transformar y automatizar la preparación de datos.

Aplicación en DataMart 360

Se implementó un flujo básico de transformación de datos que permitió automatizar tareas repetitivas de limpieza y consolidación.

Temas trabajados

- Importación mediante Power Query.
- Transformación de columnas.
- Cambio de tipos de datos.
- Combinación de consultas.
- Anexado de tablas.
- Actualización automática.

Importación y conexión de datos

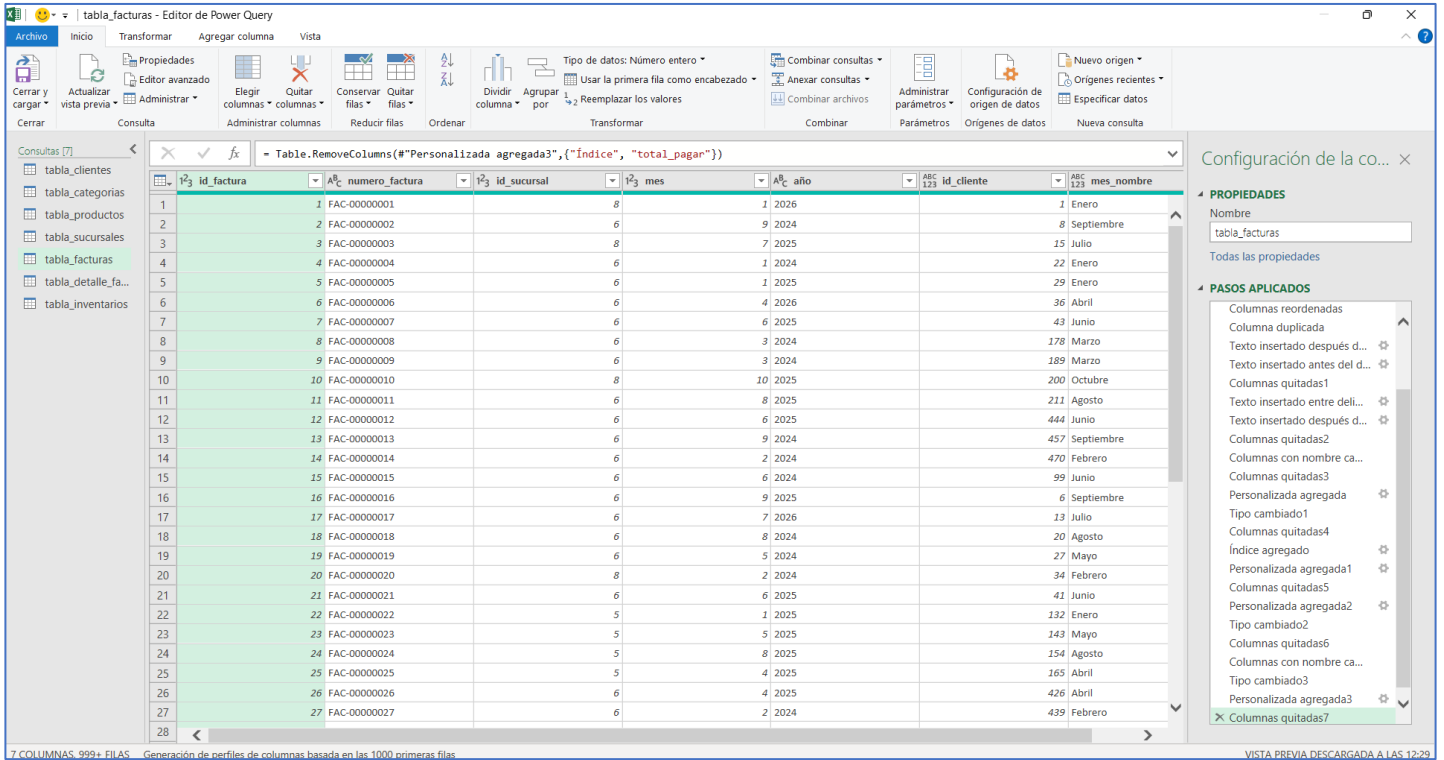


Table: RemoveColumns(#"Personalizada agregada3",{"índice", "total_pagar"})

	id_factura	numero_factura	id_sucursal	mes	año	id_cliente	mes_nombre
1	1	FAC-00000001		8	1 2026		1 Enero
2	2	FAC-00000002		6	9 2024		8 Septiembre
3	3	FAC-00000003		8	7 2025		15 Julio
4	4	FAC-00000004		6	1 2024		22 Enero
5	5	FAC-00000005		6	1 2025		29 Enero
6	6	FAC-00000006		6	4 2026		36 Abril
7	7	FAC-00000007		6	6 2025		43 Junio
8	8	FAC-00000008		6	3 2024		178 Marzo
9	9	FAC-00000009		6	3 2024		189 Marzo
10	10	FAC-00000010		8	10 2025		200 Octubre
11	11	FAC-00000011		6	8 2025		211 Agosto
12	12	FAC-00000012		6	6 2025		444 Junio
13	13	FAC-00000013		6	9 2024		457 Septiembre
14	14	FAC-00000014		6	2 2024		470 Febrero
15	15	FAC-00000015		6	6 2024		99 Junio
16	16	FAC-00000016		6	9 2025		6 Septiembre
17	17	FAC-00000017		6	7 2026		13 Julio
18	18	FAC-00000018		6	8 2024		20 Agosto
19	19	FAC-00000019		6	5 2024		27 Mayo
20	20	FAC-00000020		8	2 2024		34 Febrero
21	21	FAC-00000021		6	6 2025		41 Junio
22	22	FAC-00000022		5	1 2025		132 Enero
23	23	FAC-00000023		5	5 2025		143 Mayo
24	24	FAC-00000024		5	8 2025		154 Agosto
25	25	FAC-00000025		5	4 2025		165 Abril
26	26	FAC-00000026		6	4 2025		426 Abril
27	27	FAC-00000027		6	2 2024		439 Febrero

7 COLUMNAS, 999+ FILAS Generación de perfiles de columnas basada en las 1000 primeras filas

VISTA PREVIA DESCARGADA A LAS 12:29

Configuración de la columna

PROPIEDADES

Nombre: tabla_facturas

Todas las propiedades

PASOS APLICADOS

- Columnas reordenadas
- Columna duplicada
- Texto insertado después d...
- Texto insertado antes del d...
- Columnas quitadas1
- Texto insertado entre del...
- Texto insertado después d...
- Columnas quitadas2
- Columnas con nombre ca...
- Columnas quitadas3
- Personalizada agregada
- Tipo cambiado1
- Columnas quitadas4
- Índice agregado
- Personalizada agregada1
- Columnas quitadas5
- Personalizada agregada2
- Tipo cambiado2
- Columnas quitadas6
- Columnas con nombre ca...
- Tipo cambiado3
- Personalizada agregada3
- Columnas quitadas7

7.4 Funciones de búsqueda y referencia

¿Qué son?

Funciones que permiten localizar información relacionada entre diferentes tablas.

Aplicación en DataMart 360

Se utilizaron para relacionar productos, categorías, clientes y facturas.

Temas trabajados

- BUSCARV.
- BUSCARX.
- INDEX.
- MATCH.
- INDEX + MATCH.
- SI.ERROR.

7.5 Funciones de análisis

¿Qué son?

Funciones destinadas al cálculo de indicadores, consolidación de datos y análisis de información.

Aplicación en DataMart 360

Se calcularon ventas por sucursal, ventas por categoría, cantidad de facturas y otros indicadores comerciales.

Temas trabajados

- SUMAR.SI.
- SUMAR.SI.CONJUNTO.
- CONTAR.SI.
- CONTAR.SI.CONJUNTO.
- PROMEDIO.SI.
- PROMEDIO.SI.CONJUNTO.

7.6 Análisis mediante tablas dinámicas

¿Qué son?

Herramienta de Excel para resumir, agrupar y analizar grandes volúmenes de información.

Aplicación en DataMart 360

Se construyeron reportes dinámicos para el análisis de ventas, inventario y desempeño comercial.

Temas trabajados

- Creación de tablas dinámicas.
- Agrupación por fechas.
- Ventas por sucursal.
- Ventas por categoría.
- Productos más vendidos.
- Ranking de clientes.
- Campos calculados.

Tablas dinámicas

VENTAS POR MES		VENTAS POR CATEGORÍAS		TOP 10 PRODUCTOS POR INGRESO		TOP 10 CLIENTES POR VENTAS		STOCK CRÍTICO					
Año y mes	Ventas totales	Categoría	Ventas Totales	Productos	Ventas Totales	Clientes	Suma de total_p	Productos	Stock	To	Suma de stock_minimo	Productos	Stock
2024		Electrónica	\$147,547.27	Licudadora de Vaso de Vidrio Oster 15L	\$20,528.00	Juan Sanchez Delgado	\$10,634.83	Resma de Papel Bond A4 Report 500 Hojas	230,166				1
ene	\$23,989.51	Hogar	\$115,737.90	Collar Antipulgas para Perros Seresto	\$17,160.00	Silvia Sanchez Delgad	\$9,897.61	Aros de Cebolla Rebozados McCains 400g	143,347				8
feb	\$23,624.58	Carnes	\$67,568.30	Audifonos Bluetooth Inalámbricos JBL Tune	\$16,682.00	Camila Vega Paredes	\$3,737.66	Aguardiente Zhumir Seco 750ml	727,316				1
mar	\$25,682.34	Mascotas	\$53,541.45	Cafetera de Filtro Eléctrica Oster 12 Tazas	\$15,705.00	Andrea Ortiz Solis	\$3,736.91	Cinta Adhesiva Transparente Scotch 18mx12mm x2	203,477				8
abr	\$27,202.89	Bebés	\$47,322.95	Router inalámbrico Smart 11P-Link Archer	\$13,938.00	Valeria Morales Mora	\$9,537.83	Ganchos de Ropa Plásticos pack x12	155,542				6
may	\$26,945.26	Líquidos	\$44,386.50	Olla de Presión de Aluminio Umco 4L	\$13,172.10	Juan Cardenas Castro	\$9,473.33	Hamburguesas de Pollo Fritz x4	266,629				1
jun	\$25,010.03	Congelados	\$32,898.15	Smartwatch Deportivo Digital Band 6	\$11,840.40	Diego Cardenas Castro	\$9,450.34	Organizador de Zapatos Plástico Armable	180,95				1
jul	\$25,725.93	Embutidos	\$28,886.62	Fórmula de Crecimiento Progress Gold 3 900g	\$11,221.00	Camila Silva Diaz	\$9,350.95	Resaltadores de Texto Stabilo Boss pack x4	87,202				8
ago	\$23,921.26	Aseo Personal	\$28,886.25	Alimento Perro Cachorro ProPlan 3kg	\$11,005.80	Camila Cabrera Perez	\$9,121.18	Sacapuntas con Depósito Plástico x2	209,054				1
sept	\$22,275.63	Bebidas	\$27,773.55	Fórmula Infantil Nan Optipro 1 Pro 800g	\$10,147.50	Luis Ramirez Viteri	\$9,031.26	Adaptador DTG USB a Tipo C Metálico	120,396				1
oct	\$22,796.17							Agua Mineral Güitig 1.5L	2042,691				10
nov	\$25,510.19							Agua Purificada Dasani 1.5L	1058,524				8
dic	\$23,479.65							Agua Tónica Fioravanti 1L	1261,238				10
2025								Aji para Seco en pasta 200g	1499,433				10
ene	\$24,886.29							Aspirador Nasal de Goma Infantil	198,579				10
feb	\$20,561.69							Audifonos Bluetooth Inalámbricos JBL Tune	239,161				8
mar	\$23,897.91							Batería Portátil Powerbank Adala 10000mAh	229,216				6
abr	\$24,826.57							Bebedero y Comedero Plástico Mascotas	564,891				8
may	\$26,456.38							Bolígrafo Bic Cristal Azul pack x4	122,323				8
jun	\$29,061.72							Bolsas de Basura Fundasol Grande x10	466,93				10
jul	\$25,089.63							Borrador de Queso Faber-Castell pack x2	323,588				10
ago	\$28,141.18							Cafetera de Filtro Eléctrica Oster 12 Tazas	311,37				10
sept	\$25,001.37							Cámara Web USB HD con Micrófono	249,786				1
oct	\$25,496.98							Camarón Pelado y Desvenado 500g	240,912				8
nov	\$24,323.58							Carne de Pavo Troceada Soprano 1kg	162,006				6
dic	\$22,367.91							Carne Molida Especial Res 1kg	297,202				8
2026								Carne para Estofado de Res 1kg	286,334				1
ene	\$26,307.49							Cartulina Escolar de Colores A4 pack x20	302,08				8
feb	\$25,021.06							Chuleta de Cerdo Mr. Chancho 1kg	325,148				6
mar	\$26,027.82							Chupón de Silicona Ortodóntico Chico x2	113,382				10
abr	\$25,216.89							Clips de Oficina Metálicos Standard caja x100	332,616				1
may	\$27,114.81							Coca-Cola 2L	1345,024				10
jun	\$26,003.18							Colador de Acero Inoxidable de Cocina	195,257				10
jul	\$26,543.77							Colgate Total 12 Crema Dental 150g	345,766				8
ago	\$13,364.78							Corrector Líquido tipo Lápiz Liquid Paper	326,347				10
Total general	\$791,873.65							Costilla de Cerdo BBQ Mr. Chancho 1kg	236,547				6
								Crema Antipañalitis Destin Original 113g	211,596				8
								Crema de Leche La Lechera 250g	1243,598				8
								Cuchillo de Cocina Tramontina Profesional 8"	176,331				10
								Dedos de Queso Mozzarella Fritz 350g	100,13				8
								Detergente Deja Limón Líquido 3L	703,883				8
								Filete de Corvina Fresco 1kg	290,474				10
								Filete de Salmón Importado 500g	248,341				8
								Foco LED Ledlighter, Viscosi, Secad, Doble	169,067				10

7.7 Visualización y exploración de datos

¿Qué es?

Proceso de representación gráfica de la información para facilitar el análisis y la interpretación.

Aplicación en DataMart 360

Se desarrollaron visualizaciones para analizar tendencias de ventas y comportamiento del negocio.

Temas trabajados

- Gráficos dinámicos.
- Segmentación de datos.
- Líneas de tiempo.
- Filtros avanzados.
- Ordenamientos personalizados.

7.8 Formato condicional

¿Qué es?

Herramienta que permite resaltar información automáticamente según reglas definidas.

Aplicación en DataMart 360

Se implementaron alertas visuales para inventario, ventas y desempeño de productos.

Temas trabajados

- Escalas de color.
- Barras de datos.
- Conjuntos de iconos.
- Alertas de bajo inventario.
- Productos con mayor rotación.
- Ventas por encima del objetivo.

7.9 Validación de datos

¿Qué es?

Mecanismo que controla el tipo de información que puede ingresar el usuario en una hoja de cálculo.

Aplicación en DataMart 360

Se diseñaron formularios y plantillas de captura con controles para evitar errores de digitación.

Temas trabajados

- Listas desplegables.
- Restricción de fechas.
- Restricción numérica.
- Mensajes de entrada.
- Mensajes de error.
- Control de valores permitidos.

7.10 Dashboard ejecutivo en Excel

¿Qué es?

Panel interactivo que consolida indicadores clave del negocio para apoyar la toma de decisiones.

Aplicación en DataMart 360

Se construyó un dashboard ejecutivo utilizando los datos exportados desde PostgreSQL y transformados mediante Excel.

Indicadores incluidos

- Ventas por mes
- Ventas por sucursal
- Ventas por categoría
- Top 10 productos
- Cantidad vendida por producto
- Ventas por cliente
- Clientes por provincia
- Inventario por sucursal
- Inventario por categoría
- Productos con stock crítico
- Ventas por provincia

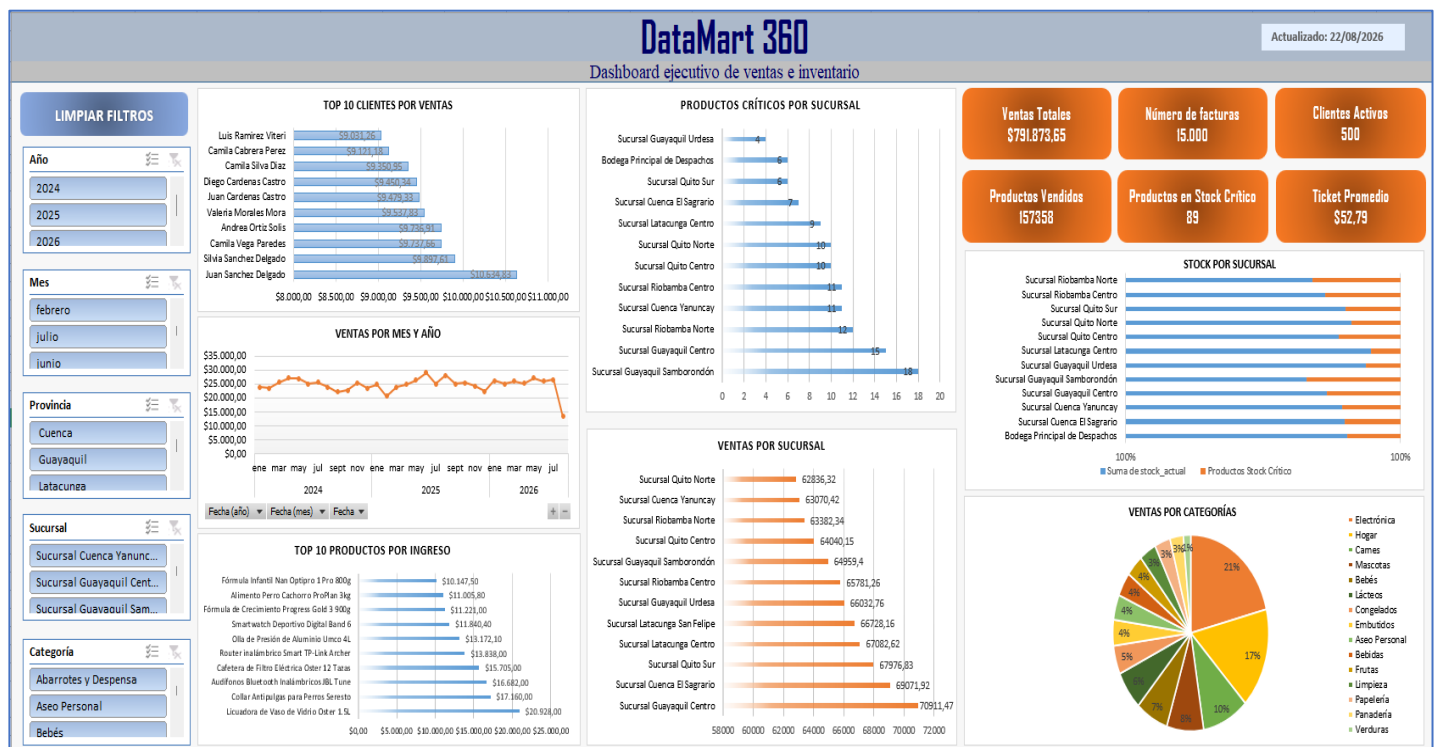
Elementos interactivos

- Segmentación de datos.
- Líneas de tiempo.
- Gráficos dinámicos.
- Indicadores visuales.

Resultado obtenido

Un panel ejecutivo completamente interactivo que permite analizar el desempeño comercial de **DataMart 360**.

Dashboard final



Conclusiones del módulo

En este módulo se desarrolló un flujo completo de análisis de datos utilizando Excel como herramienta de Business Intelligence. La integración con PostgreSQL mediante Power Query permitió automatizar la preparación de datos, construir un modelo relacional y generar un dashboard ejecutivo completamente interactivo.

El proyecto DataMart 360 demuestra la capacidad de transformar datos operacionales en información analítica útil para la toma de decisiones, integrando procesos ETL, modelado de datos, análisis mediante tablas dinámicas y visualización ejecutiva en una solución profesional desarrollada íntegramente en Excel.

MÓDULO 8

POWER BI

Introducción

El presente módulo corresponde a la etapa de visualización y análisis empresarial del proyecto DataMart 360, desarrollado para una cadena de supermercados.

El objetivo de esta fase fue transformar la información almacenada en PostgreSQL y preparada durante las etapas anteriores en un modelo analítico capaz de generar indicadores, visualizaciones y análisis orientados a la toma de decisiones.

Power BI fue utilizado como herramienta de Business Intelligence para integrar los datos, construir medidas DAX, establecer relaciones entre las entidades y desarrollar un dashboard interactivo dividido por dominios de negocio.

El resultado final no se planteó como un dashboard exclusivamente visual, sino como una herramienta de análisis que permite responder preguntas relacionadas con ventas, clientes, productos, inventarios, compras, proveedores, sucursales y operaciones.

Objetivos del módulo

Objetivo general

Construir un dashboard empresarial en Power BI que permita analizar el comportamiento comercial y operativo de DataMart 360 mediante indicadores y visualizaciones interactivas.

Objetivos específicos

- Conectar Power BI con la base de datos del proyecto.
- Incorporar las tablas necesarias para el análisis.
- Validar las relaciones entre las tablas.
- Construir un modelo de datos funcional.
- Crear medidas DAX para los principales indicadores.
- Analizar ventas, clientes, productos, inventarios y compras.
- Incorporar información de proveedores y sucursales.
- Analizar los movimientos entre sucursales.
- Crear filtros interactivos por dominio.
- Validar los resultados obtenidos en Power BI contra PostgreSQL.
- Diseñar un dashboard organizado en páginas temáticas.

- Evitar duplicidad de información entre visualizaciones.
- Generar una herramienta orientada al análisis y la toma de decisiones.

Fuente de datos

La fuente principal de información utilizada para el desarrollo del dashboard fue la base de datos DataMart 360, implementada previamente en PostgreSQL.

El modelo contiene tablas transaccionales, tablas maestras y tablas de relación.

Entre las principales tablas utilizadas en Power BI se encuentran:

Ventas

- facturas
- detalles_facturas

Clientes

- clientes
- tarjetas_fidelizacion

Productos

- productos
- categorias
- productos_promociones
- promociones

Inventarios

- inventarios

Compras

- compras
- detalles_compras
- proveedores

Sucursales y operaciones

- sucursales
- despachos_sucursales
- detalles_despachos

Tablas auxiliares

- `calendario`
- `metodos_pago`

Otras tablas administrativas existentes en la base de datos, como `empleados`, `usuarios_sistema` y `auditoria_sistema`, forman parte del sistema general, aunque no todas fueron incorporadas como elementos visuales principales del dashboard.

Modelo de datos en Power BI

El modelo de Power BI fue construido respetando las relaciones definidas previamente en PostgreSQL.

Las principales relaciones utilizadas son:

- facturas → clientes
- facturas → metodos_pago
- facturas → sucursales
- facturas → calendario
- detalles_facturas → facturas
- detalles_facturas → productos
- productos → categorias
- inventarios → productos
- inventarios → sucursales
- compras → proveedores
- compras → sucursales
- detalles_compras → compras
- detalles_compras → productos
- despachos_sucursales → sucursales
- detalles_despachos → despachos_sucursales
- detalles_despachos → productos
- productos_promociones → productos
- productos_promociones → promociones
- tarjetas_fidelizacion → clientes

La relación principal para el análisis de ventas por sucursal corresponde a:

`facturas[id_sucursal] → sucursales[id_sucursal]`

Permitiendo utilizar el nombre de la sucursal como dimensión y `total_pagar` como valor monetario.

Validación del modelo

Durante el desarrollo se realizaron diferentes validaciones para comprobar que los resultados de Power BI fueran coherentes con PostgreSQL.

Uno de los principales controles se realizó sobre las ventas y el número de facturas.

La base contiene aproximadamente:

- **15.000 facturas**
- **792 mil de ventas totales**

La distribución de las facturas y ventas por sucursal también fue comprobada directamente en PostgreSQL.

Esta validación fue importante porque inicialmente algunos visuales mostraban el mismo valor para diferentes sucursales, lo que permitió detectar problemas relacionados con el contexto de filtro y la configuración del modelo.

Después de revisar las relaciones y el contexto de los visuales, se verificó que las sucursales presentaran valores diferenciados.

Medidas DAX principales

Para centralizar los cálculos utilizados por el dashboard se creó una tabla de medidas denominada medidas.

Entre las medidas principales se encuentran:

Clientes activos

```
Cientes Activos =  
DISTINCTCOUNT(  
    facturas[id_cliente]  
)
```

Permite determinar la cantidad de clientes que poseen al menos una factura registrada.

Número de facturas

```
Numero Facturas =  
DISTINCTCOUNT(  
    facturas[id_factura]  
)
```

Cuenta las facturas únicas utilizadas como transacciones de venta.

Productos vendidos

```
Productos Vendidos =  
SUM(  
    detalles_facturas[cantidad]  
)
```

Representa la cantidad total de unidades vendidas.

Sucursales activas

```
Sucursales Activas =  
DISTINCTCOUNT(  
    sucursales[id_sucursal]  
)
```

Determina la cantidad de sucursales disponibles en el modelo.

Stock total

```
Stock Total =  
SUM(  
    inventarios[stock_actual]  
)
```

Calcula las unidades disponibles registradas en inventario.

Productos stock crítico

```
Productos Stock Crítico =  
CALCULATE(  
    DISTINCTCOUNT(inventarios[id_producto]),  
    FILTER(  
        inventarios,  
        inventarios[stock_actual]  
            <= RELATED(productos[stock_minimo_alerta])  
    )  
)
```

Identifica los productos cuyo stock actual es igual o inferior al nivel mínimo establecido.

Stock crítico

```
Stock Crítico =  
CALCULATE(  
    [Stock Total],  
    FILTER(  
        inventarios,  
        inventarios[stock_actual]  
            <= RELATED(productos[stock_minimo_alerta])  
    )  
)
```

)

Calcula las unidades correspondientes a productos cuyo inventario se encuentra en nivel crítico.

Estado del stock

```
Estado Stock =  
VAR StockActual =  
    [Stock Total]  
  
VAR StockMinimo =  
    MAX(productos[stock_minimo_alerta])  
  
RETURN  
    SWITCH(  
        TRUE(),  
        ISBLANK(StockActual), BLANK(),  
        StockActual <= StockMinimo, "Crítico",  
        "Normal"  
    )
```

Permite clasificar el inventario en estado crítico o normal.

Margen promedio

```
Margen Promedio =  
DIVIDE(  
    [Utilidad Estimada],  
    [Ventas Totales],  
    0  
)
```

Relaciona la utilidad estimada con las ventas totales.

Crecimiento interanual

```
Crecimiento % interanual =  
VAR VentasActuales =  
    [Ventas Totales]  
  
VAR VentasPasadas =  
    [Ventas Año Anterior]  
  
RETURN  
    DIVIDE(  
        VentasActuales - VentasPasadas,  
        VentasPasadas,  
        0
```

)

Permite comparar las ventas actuales con el período anterior correspondiente.

Página 1 — Resumen Ejecutivo

La primera página funciona como una vista general del negocio.

Su objetivo es responder:

¿Cómo está funcionando el negocio en términos generales?

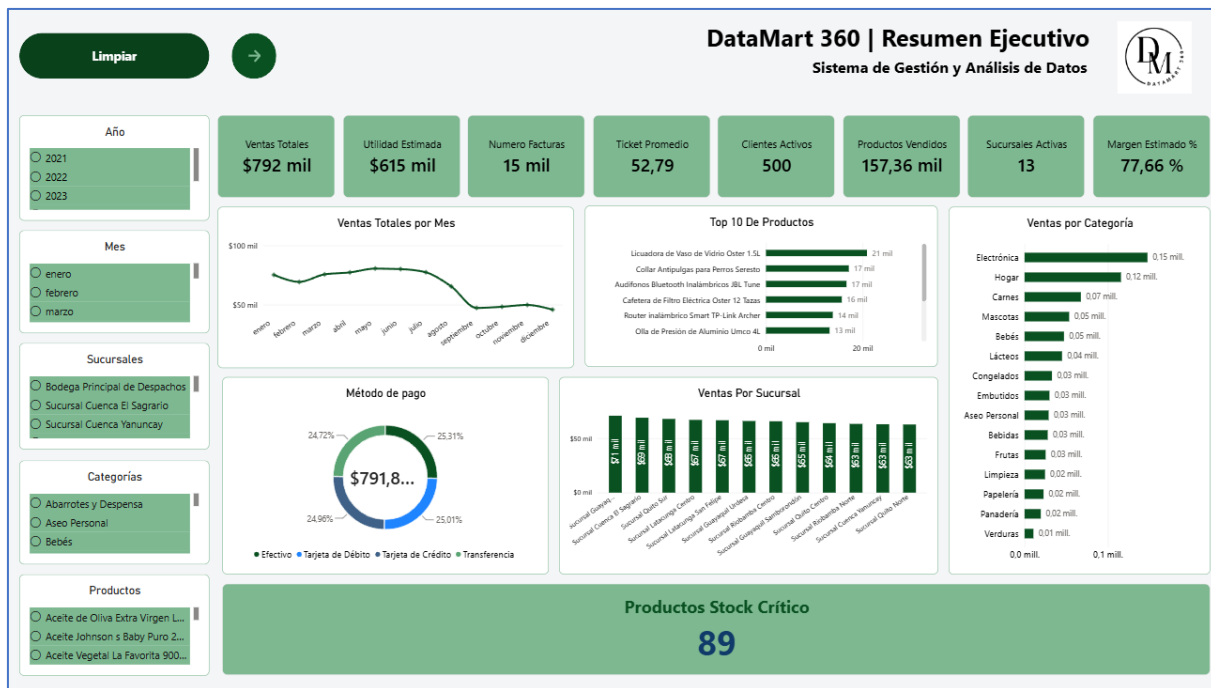
Indicadores principales

- Ventas Totales
- Utilidad Estimada
- Número de Facturas
- Ticket Promedio
- Clientes Activos
- Productos Vendidos
- Sucursales Activas
- Margen Promedio
- Productos Stock Crítico

Visualizaciones

- Ventas por período.
- Ventas por sucursal.
- Ventas por categoría.
- Productos con mayor volumen de ventas.
- Distribución por método de pago.
- Indicador de productos con stock crítico.

Esta página funciona como punto de entrada al dashboard y permite identificar rápidamente situaciones que requieren análisis posterior.



Página 2 — Análisis de Ventas

Esta página se concentra exclusivamente en el comportamiento comercial.

Pregunta de negocio

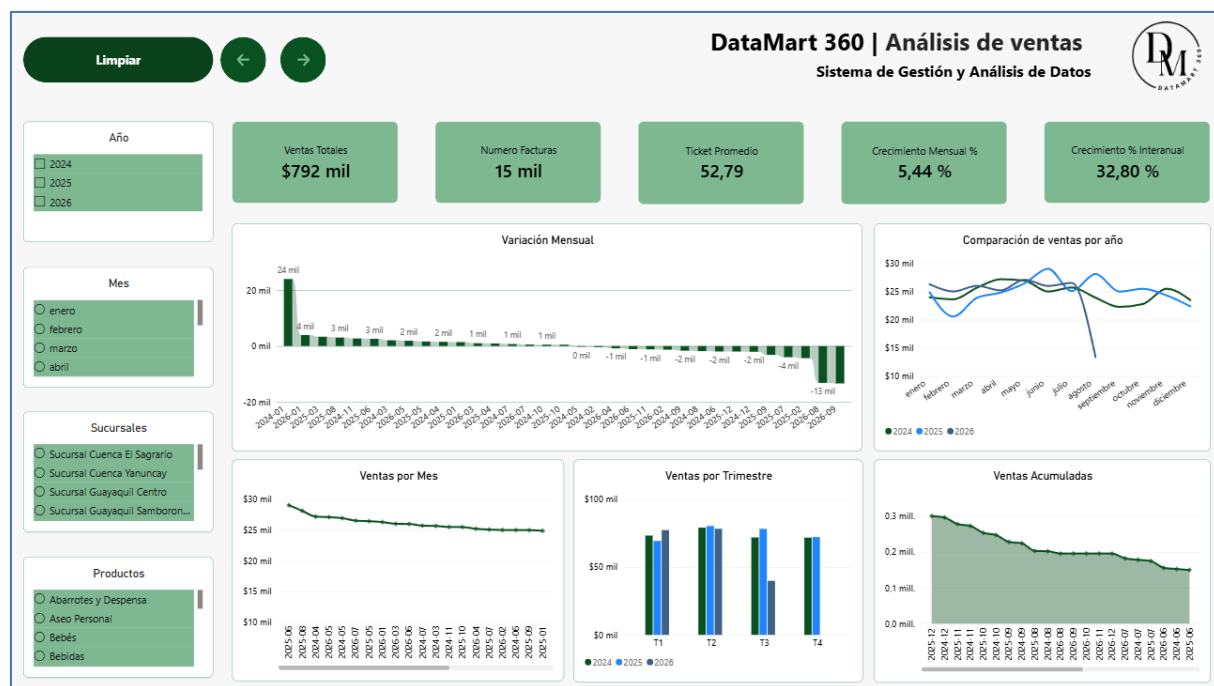
¿Cómo se comportan las ventas y cuáles son sus principales características?

Principales análisis

- Evolución de las ventas.
- Ventas por período.
- Ventas por sucursal.
- Ventas por categoría.
- Ventas por producto.
- Métodos de pago.
- Distribución del ticket.
- Comportamiento de las transacciones.

Filtros

- Fecha.
- Sucursal.
- Categoría.
- Producto.
- Método de pago, según el análisis requerido.



Página 3 — Clientes y fidelización

Esta página analiza el comportamiento de los clientes.

Pregunta de negocio

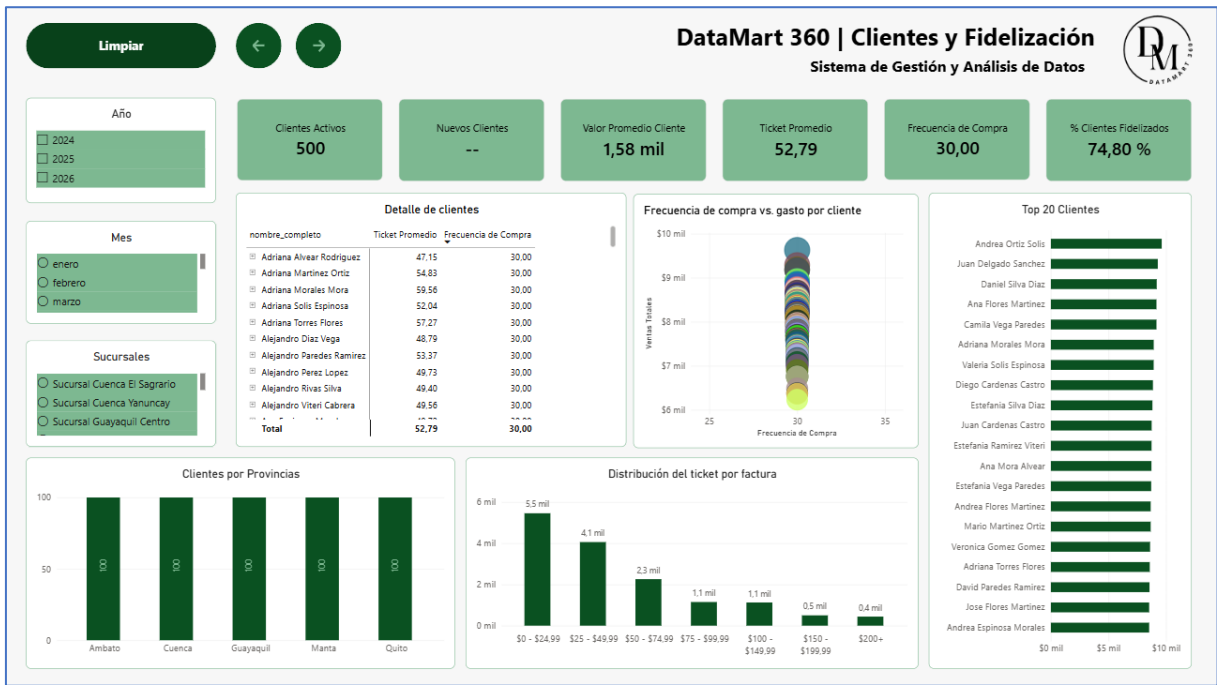
¿Quiénes son nuestros clientes y cómo se comportan?

Principales análisis

- Clientes activos.
- Clientes nuevos.
- Clientes recurrentes.
- Distribución de clientes.
- Comportamiento del ticket.
- Segmentación de clientes.
- Evolución temporal de clientes.

Se estableció que los **clientes nuevos** se identifiquen considerando clientes que presentan una primera factura dentro del período analizado.

De esta manera, la clasificación se basa en comportamiento transaccional real y no solamente en la existencia del registro en la tabla `clientes`.



Página 4 — Productos

Esta página analiza el rendimiento de los productos y categorías.

Pregunta de negocio

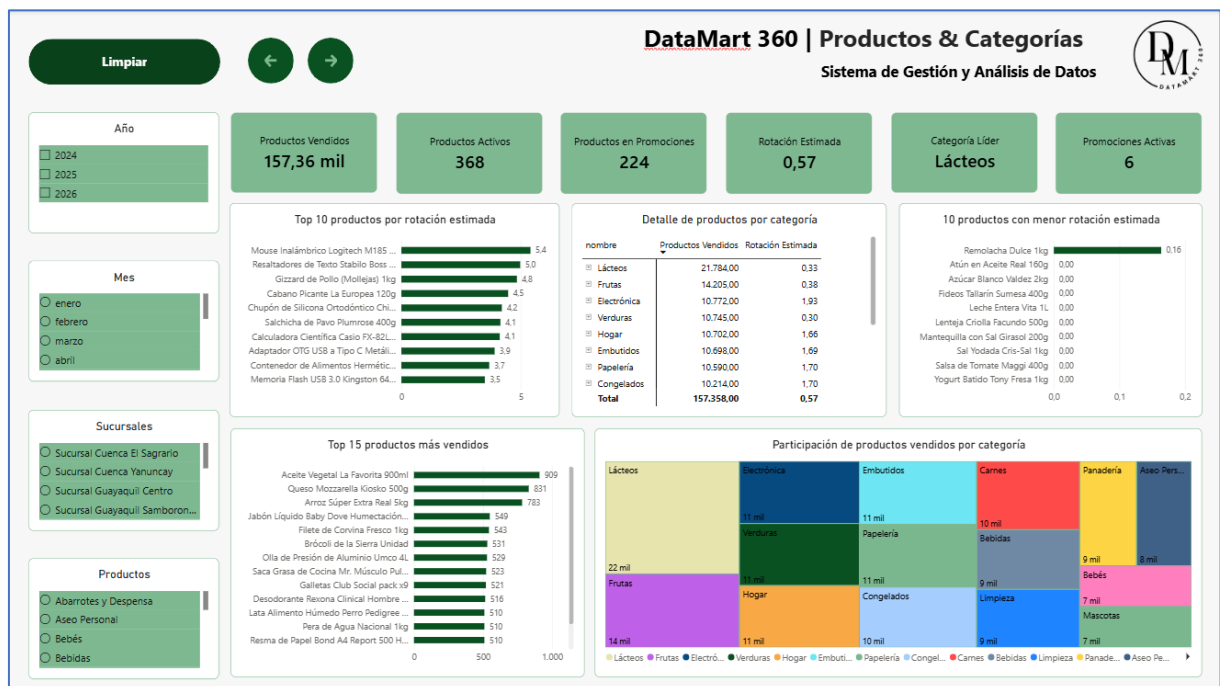
¿Qué productos tienen mayor y menor desempeño?

Principales análisis

- Productos con mayor rotación.
- Productos con menor rotación.
- Ventas por categoría.
- Productos más vendidos.
- Productos menos vendidos.
- Cantidad vendida.
- Rendimiento por categoría.

Se evitó duplicar análisis de productos entre páginas. Por ejemplo, los productos con mayor y menor rotación se concentran en esta página y no se repiten innecesariamente en la página de inventarios.

También se incorporaron análisis relacionados con promociones cuando la estructura de los datos permitió relacionar productos con campañas promocionales.



Página 5 — Inventarios y Stock

Esta página analiza la situación actual del inventario.

Pregunta de negocio

¿Qué inventario tenemos y qué productos necesitan atención?

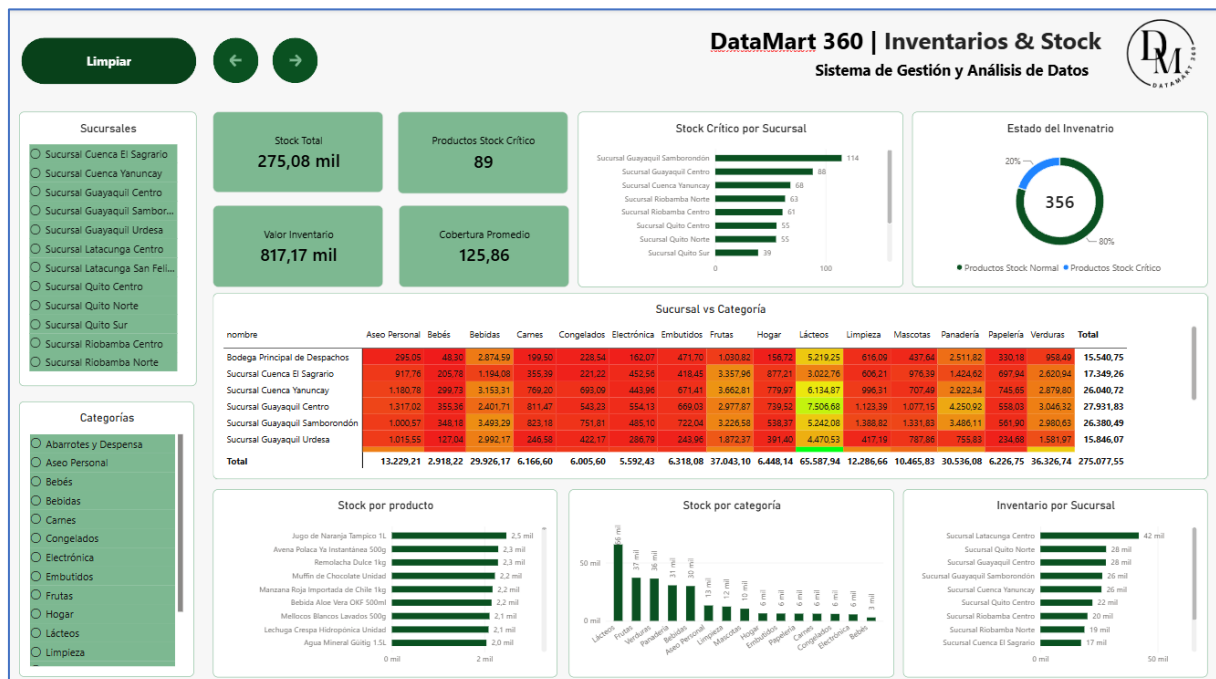
Principales visualizaciones

- Inventario por sucursal.
- Stock por categoría.
- Productos críticos.
- Heatmap de sucursal vs. categoría.
- Productos bajo mínimo.
- Stock por producto.

Filtros

Se establecieron únicamente:

- Sucursal.
- Categoría.



Página 6 — Compras y Proveedores

Esta página analiza el proceso de abastecimiento.

Pregunta de negocio

¿Cómo estamos comprando y a quién?

KPIs o tarjetas

- Compras Totales.
- Órdenes de Compra.
- Proveedores Activos.
- Costo Promedio.
- Compra Promedio.

Durante la validación se obtuvieron aproximadamente:

- **Compras Totales:** 176,94 mil.
- **Compras por detalle:** 153,86 mil.
- **Órdenes de compra:** 100.
- **Compra promedio:** 1,77 mil.
- **Costo promedio:** 7,25.

Visualizaciones

- Compras por mes.
- Compras por proveedor.
- Compras por categoría.
- Productos más comprados.
- Matriz proveedor → producto.

Indicadores

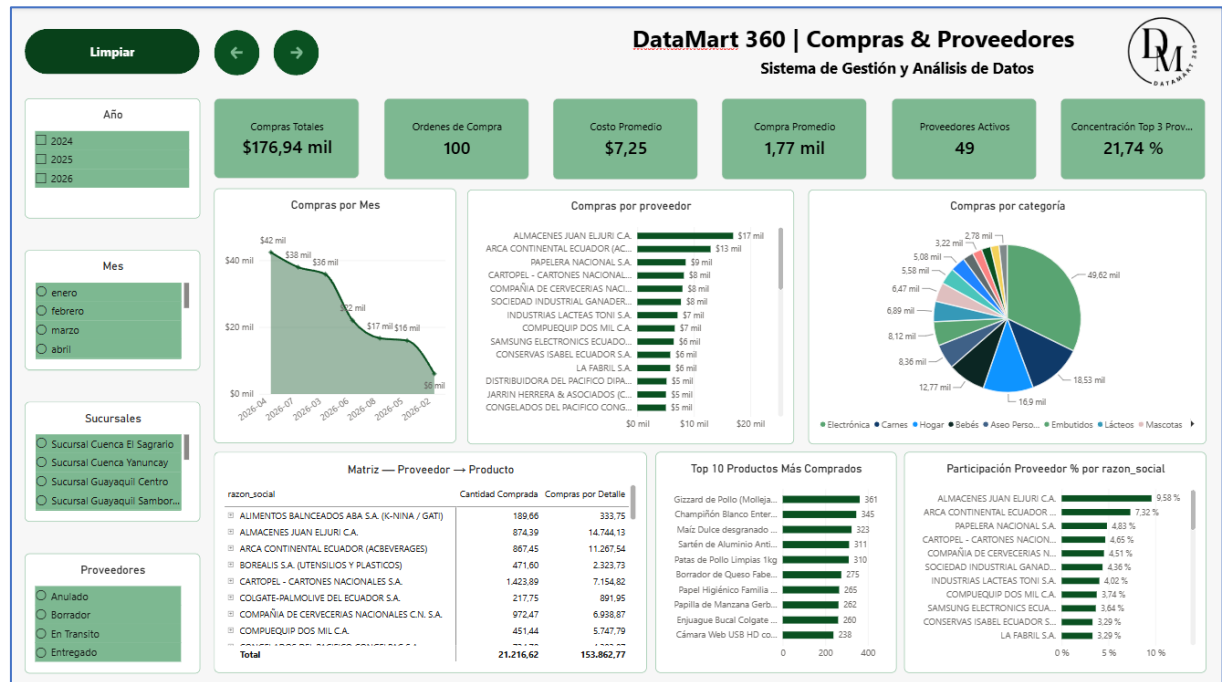
- Participación por proveedor.
- Tendencia de compras.
- Dependencia de proveedores.

Filtros

- Fecha.

- Proveedor.
- Sucursal.

La información de compras se analiza utilizando compras, detalles_compras, proveedores, productos, categorias y sucursales.



Página 7 — Operaciones y Sucursales

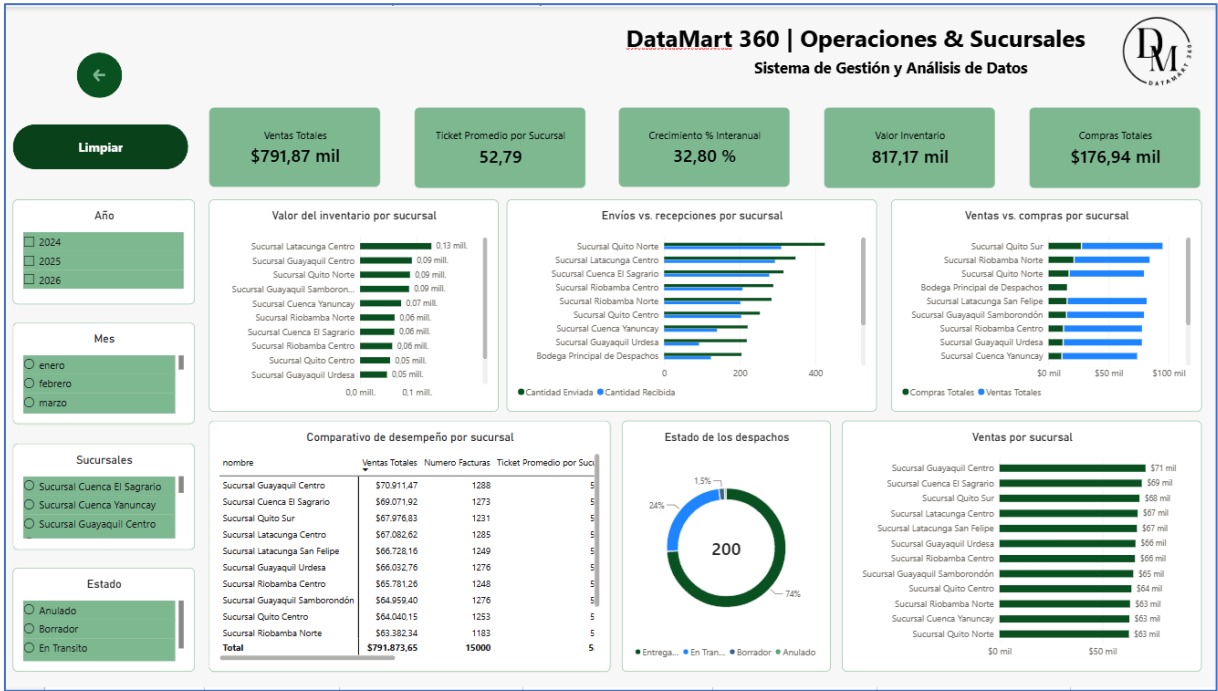
Esta página representa la vista estratégica del funcionamiento de las sucursales.

Pregunta de negocio

¿Qué sucursales tienen mejor desempeño y cómo funcionan operacionalmente?

Visualizaciones

- Ventas por sucursal
- Valor del inventario por sucursal
- Estado de los despachos
- Envíos vs. recepciones por sucursal
- Ventas vs. compras por sucursal
- Comparativo de desempeño por sucursal



Botón de limpieza de filtros

Se incorporó un botón de limpieza de filtros mediante marcadores de Power BI.

El objetivo es permitir al usuario regresar al estado inicial de los filtros sin abandonar la página actual.

Durante la implementación se identificó que un marcador global podía almacenar la página desde la cual había sido creado y provocar una navegación no deseada hacia el Resumen Ejecutivo.

Por este motivo, la estrategia adoptada consiste en mantener el comportamiento de limpieza asociado a cada página, evitando que el marcador altere la navegación del usuario.

Criterios de diseño

El dashboard fue diseñado bajo los siguientes criterios:

Claridad: Cada visualización tiene un propósito analítico definido.

No duplicidad: Se evitaron gráficos que mostraran exactamente la misma información en diferentes páginas.

Contexto de negocio

Los títulos se definieron de acuerdo con la métrica y dimensión analizada.

Ejemplos:

- Ventas por sucursal.
- Valor del inventario por sucursal.
- Estado de los despachos.
- Envíos vs. recepciones por sucursal.
- Comparativo de desempeño por sucursal.

Interactividad: Los filtros permiten explorar diferentes segmentos sin modificar el modelo de datos.

Consistencia: Se mantuvo una estructura visual uniforme entre páginas para facilitar la navegación.

Validaciones realizadas

Antes de considerar finalizado el dashboard se realizaron diferentes controles.

Validación de ventas

Se compararon los valores de Power BI con PostgreSQL.

Validación de facturas

Se verificó un volumen aproximado de: **15.000 facturas**.

Validación por sucursal

Se comprobó que las sucursales presentan valores diferentes de ventas y número de facturas.

Por ejemplo:

- Guayaquil Centro: aproximadamente 70,9 mil.
- Cuenca El Sagrario: aproximadamente 69,1 mil.
- Quito Sur: aproximadamente 68,0 mil.
- Quito Norte: aproximadamente 62,8 mil.

Esto permitió confirmar que el contexto de sucursal estaba funcionando correctamente.

Validación de compras

Se verificaron los valores generales de:

- Compras Totales.
- Compras por detalle.
- Número de órdenes.
- Compra promedio.
- Costo promedio.

Validación de despachos

Se comprobó el total de 200 despachos y su distribución por estado.

Conclusiones

La implementación de Power BI permitió transformar la base de datos DataMart 360 en una solución de Business Intelligence orientada al análisis empresarial.

El modelo desarrollado permite conectar diferentes procesos de negocio y analizar sus relaciones mediante dimensiones compartidas como:

- Fecha.
- Sucursal.
- Producto.
- Categoría.
- Cliente.
- Proveedor.

La utilización de medidas DAX permitió centralizar los principales indicadores y mantener consistencia en los cálculos.

La validación cruzada entre PostgreSQL y Power BI permitió identificar y corregir problemas relacionados con el contexto de filtros y las relaciones del modelo.

El resultado final es una herramienta interactiva que facilita la identificación de tendencias, oportunidades y situaciones que requieren atención.

Entre los principales usos del dashboard se encuentran:

- Monitorear las ventas.
- Comparar sucursales.
- Identificar productos de alta y baja rotación.
- Detectar productos con stock crítico.
- Analizar el abastecimiento.
- Evaluar proveedores.
- Comparar compras y ventas.
- Analizar el comportamiento de los clientes.
- Supervisar los movimientos entre sucursales.
- Detectar diferencias entre cantidades enviadas y recibidas.

Con este módulo se completa la etapa de Business Intelligence y visualización del proyecto DataMart 360, integrando la información generada en las etapas anteriores en una solución analítica centralizada.

Limitaciones

Como todo proyecto desarrollado con fines académicos y utilizando datos simulados, DataMart 360 presenta algunas limitaciones que deben considerarse al interpretar sus resultados.

Datos simulados

La principal limitación es que los datos utilizados fueron generados específicamente para el desarrollo del proyecto. Por esta razón, los resultados obtenidos representan el comportamiento del conjunto de datos utilizado y no pueden interpretarse como información real de una empresa.

Ausencia de conexión con sistemas productivos

La solución no está conectada a sistemas reales de facturación, inventario, compras, clientes o proveedores. Por lo tanto, la actualización de la información depende de los datos disponibles dentro del proyecto.

Alcance del análisis

Los análisis desarrollados se concentran en las áreas y variables definidas para el proyecto. No se incluyen todos los posibles factores que podrían influir en el funcionamiento de una cadena de supermercados real, como condiciones económicas, competencia, comportamiento externo del mercado, costos logísticos reales o factores macroeconómicos.

Indicadores estimados

Algunos indicadores, como la utilidad estimada, dependen de los supuestos y datos utilizados en el modelo. Por ello, no deben interpretarse como estados financieros oficiales ni como resultados contables de una empresa real.

Escenario académico

La solución fue desarrollada como proyecto académico y de demostración profesional. Aunque incorpora herramientas y procesos utilizados en entornos profesionales, no se ha implementado como sistema productivo ni ha sido sometida a las condiciones de operación, seguridad, disponibilidad y escalabilidad propias de una empresa real.

Dependencia de las herramientas utilizadas

El análisis depende de herramientas como PostgreSQL, Excel y Power BI. Cambios en las versiones, configuraciones o conexiones entre estas herramientas podrían requerir ajustes en los procesos desarrollados.

Escalabilidad

El volumen de datos utilizado permite demostrar las funcionalidades principales del proyecto, pero no representa necesariamente el comportamiento de una base de datos empresarial con millones de registros y múltiples usuarios concurrentes. Por ello, aspectos como optimización avanzada, alta disponibilidad y arquitectura distribuida quedan fuera del alcance de esta versión.

Fuentes de apoyo y herramientas utilizadas

Durante el desarrollo del proyecto se utilizaron recursos de documentación técnica, materiales educativos y herramientas digitales como apoyo para el aprendizaje, investigación y resolución de problemas.

Entre las principales fuentes y herramientas de apoyo se encuentran:

- **Microsoft Excel:** documentación y recursos de aprendizaje relacionados con funciones, Power Query, Power Pivot y tablas dinámicas.
- **Microsoft Power BI:** documentación y recursos de aprendizaje relacionados con modelado de datos, visualizaciones y DAX.
- **PostgreSQL:** documentación y recursos de consulta relacionados con SQL, bases de datos y funcionalidades del sistema gestor.
- **Draw.io:** herramienta utilizada para el diseño de diagramas y modelos.
- **ChatGPT — OpenAI:** utilizado como herramienta de apoyo para investigación, aprendizaje, resolución de dudas técnicas y exploración de alternativas.
- **Gemini — Google:** utilizado como herramienta complementaria de apoyo durante el aprendizaje y desarrollo del proyecto.
- **Otros recursos educativos:** materiales técnicos consultados durante las diferentes etapas del proyecto.

Declaración sobre el uso de herramientas de inteligencia artificial

Durante el desarrollo de DataMart 360 se utilizaron herramientas de inteligencia artificial como apoyo complementario en el proceso de aprendizaje y desarrollo del proyecto.

Se utilizaron principalmente ChatGPT (OpenAI) y Gemini (Google) para actividades como:

- Investigación y aclaración de conceptos técnicos.
- Resolución de dudas relacionadas con SQL, PostgreSQL, Excel, Power Query, DAX y Power BI.
- Exploración de alternativas para resolver problemas técnicos.
- Revisión y comprensión de errores durante el desarrollo.
- Apoyo en la organización y documentación de determinadas etapas del proyecto.
- Las herramientas de inteligencia artificial fueron utilizadas como recursos de apoyo, no como sustitutos del trabajo realizado en el proyecto.

- La definición del problema, el diseño de la solución, la estructura de la base de datos, la implementación, la selección de herramientas, la elaboración de consultas, la transformación y análisis de los datos, la construcción de los modelos y dashboards, así como la validación de los resultados, fueron desarrollados y revisados por la autora.
- El uso de estas herramientas formó parte del proceso de aprendizaje y permitió complementar la investigación y resolver dificultades técnicas durante el desarrollo de la solución.